Dossier de TP : GitOps & CI/CD

Compte-rendu des travaux pratiques de la semaine

Nom et prénom :	Johan Colas
Formation :	B2 Informatique - Cours GitOps & CI/CD
Date :	Novembre 2026
Dépôt applicatif :	https://github.com/jocomte/tasks-api
Dépôt de config (GitOps) :	https://github.com/jocomte/tasks-config

Sommaire

1. TP1 : Première CI avec GitHub Actions

1.1 Objectif

1.2 Déroulement du TP

1.3 Captures d'écran (TP1-1 à TP1-7)

1.4 Réponses aux questions

1.5 Difficultés rencontrées

2. TP2 : Une CI solide

2.1 Objectif

2.2 Déroulement du TP

2.3 Captures d'écran (TP2-1 à TP2-11)

2.4 Réponses aux questions

2.5 Difficultés rencontrées

3. TP3 : Docker et livraison continue

3.1 Objectifs et démarche

3.2 Tableau de synthèse des captures

3.3 Captures d'écran (TP3-1 à TP3-8)

3.4 Réponses aux questions

3.5 Difficultés rencontrées

4. TP4 : GitOps

5. Bilan personnel

1. TP1 : Première CI avec GitHub Actions

1.1 Objectif

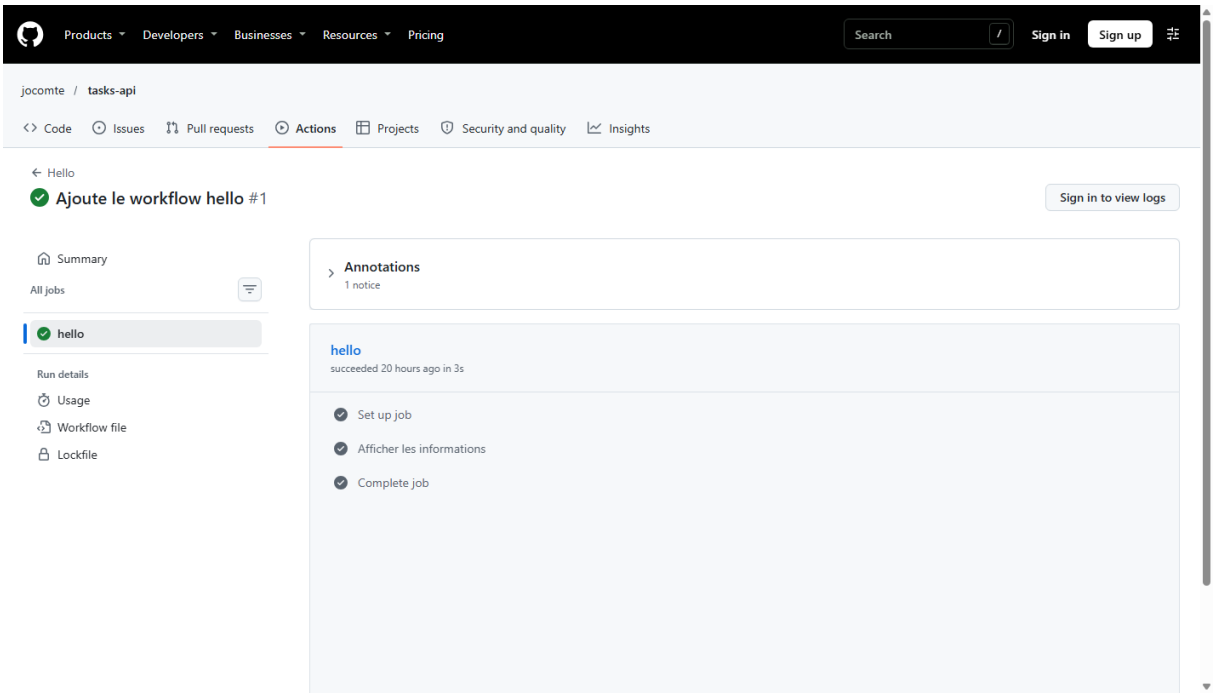
L'objectif de ce premier TP était de mettre en place notre premier pipeline d'intégration continue avec GitHub Actions. Le but était de faire en sorte que chaque push sur le dépôt déclenche automatiquement les tests du projet, d'apprendre à lire les logs d'exécution (notamment quand un test échoue), et de sécuriser la branche principale en conditionnant la fusion d'une pull request au passage des tests.

1.2 Déroulement du TP

- **Création de la branche de travail :** J'ai créé la branche tp1 avec la commande git switch -c tp1 pour travailler de manière isolée et ne pas modifier directement la branche main.
- **Workflow initial (hello.yml) :** J'ai configuré un premier fichier .github/workflows/hello.yml déclenché à chaque push sur Ubuntu. Il affiche un message ainsi que la branche et l'auteur.
- **Workflow de tests (ci.yml) :** J'ai créé le workflow ci.yml qui installe Python 3.12, installe les dépendances avec pip, puis exécute pytest.
- **Test d'échec et correction :** J'ai réduit volontairement MAX_TITLE_LENGTH à 5 dans app/logic.py pour observer le run rouge. J'ai ensuite rétabli la valeur à 80 pour repasser au vert.
- **Pull request et badge :** J'ai ouvert une pull request vers main, vérifié les statuts de tests, fusionné la PR et ajouté le badge de statut dans le README.md.

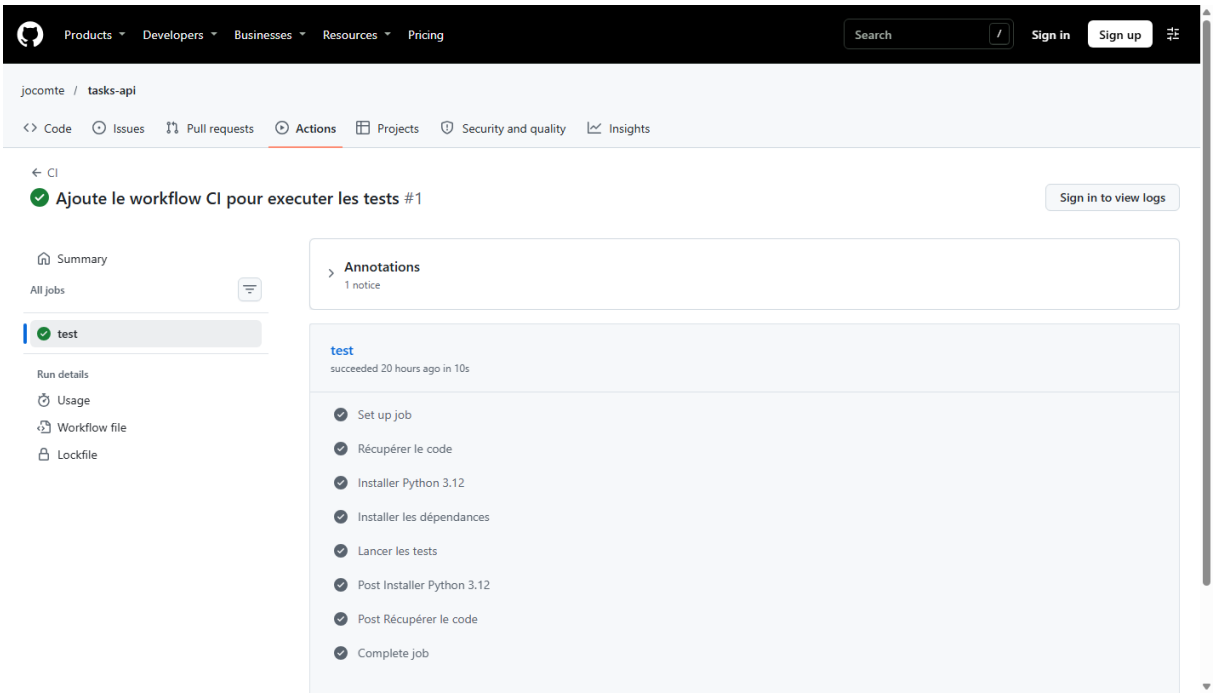
1.3 Captures d'écran du TP1

Capture TP1-1 : Run « Hello » réussi avec étape dépliée



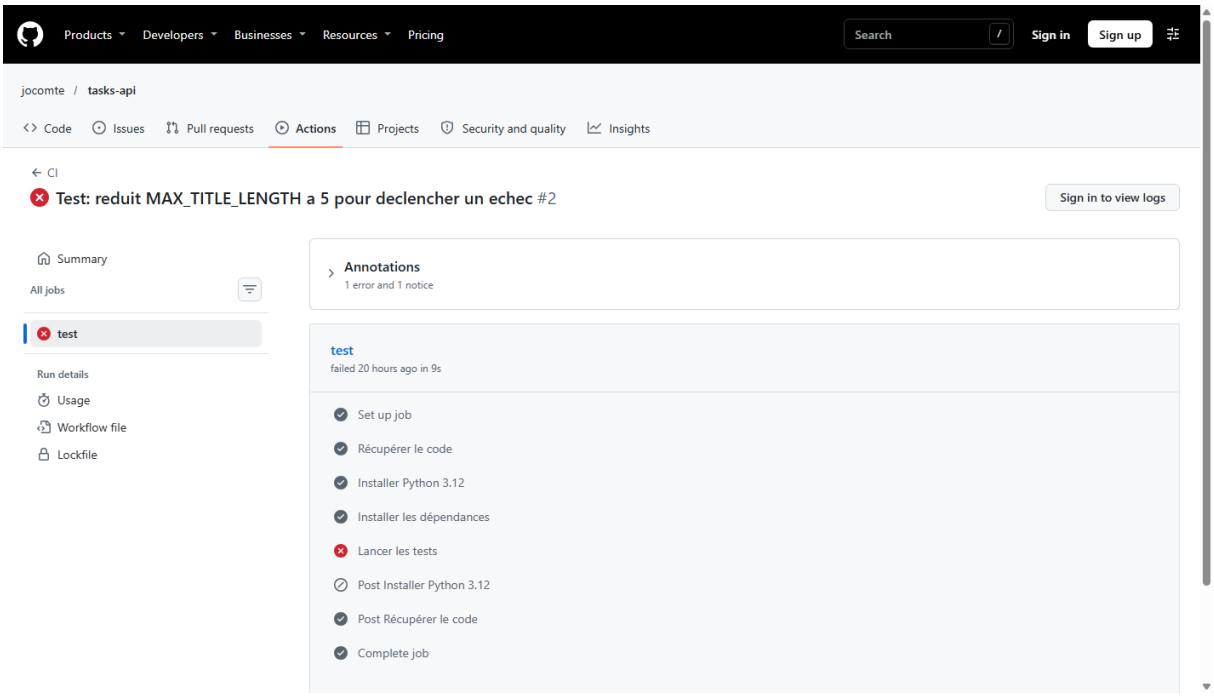
Légende TP1-1 : Le run Hello est validé sur la branche tp1. L'étape « Afficher les informations » est dépliée et indique la branche tp1 et l'auteur jocomte.

Capture TP1-2 : Run ci.yml réussi avec les étapes visibles



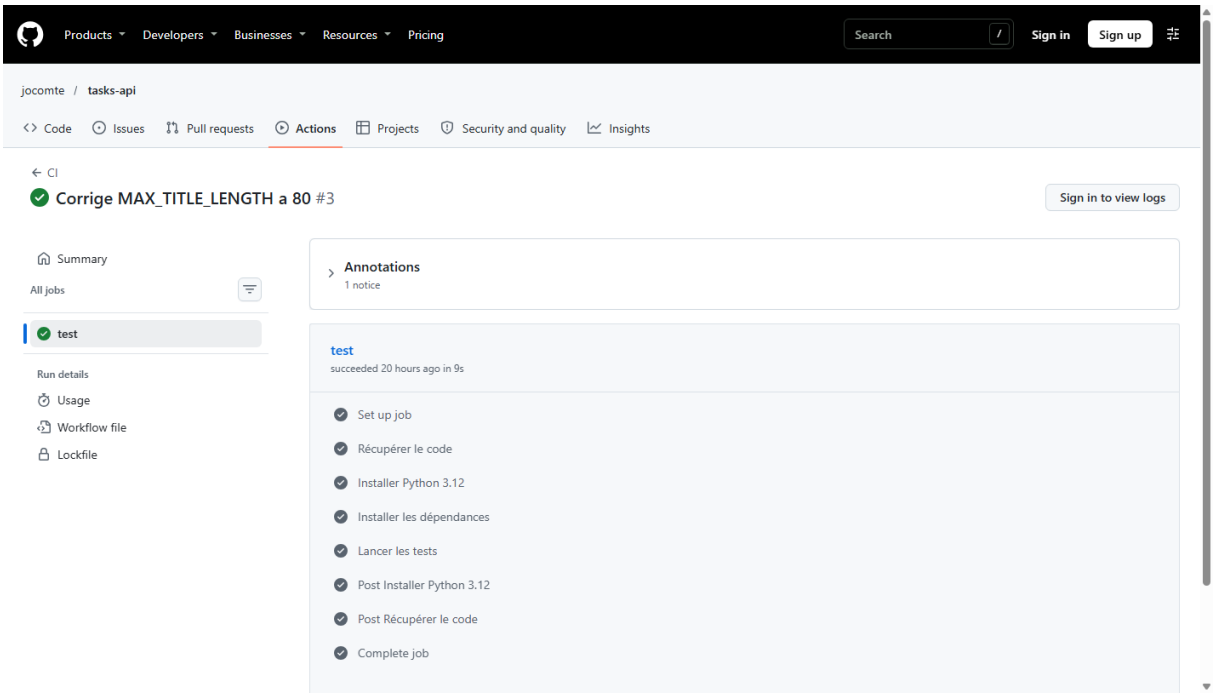
Légende TP1-2 : Le job test du fichier ci.yml a réussi avec les étapes de checkout, configuration Python 3.12, dépendances et pytest.

Capture TP1-3 : Run en échec (rouge) avec le message d'erreur



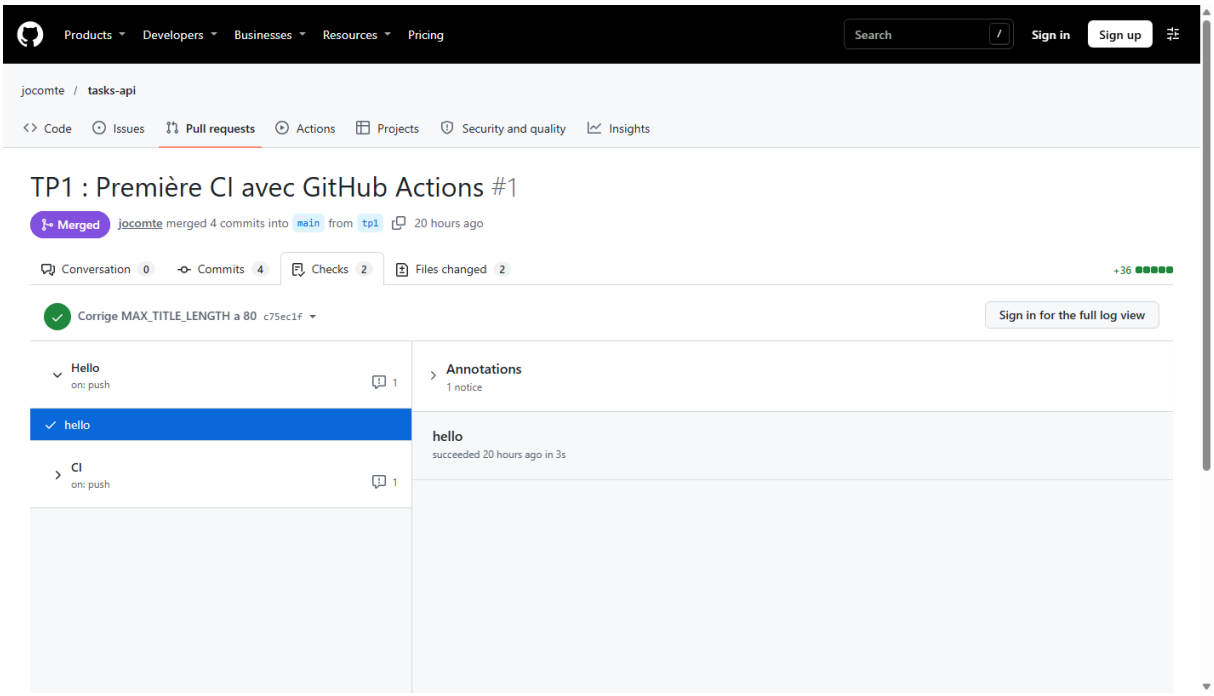
Légende TP1-3 : Échec de la CI provoqué par le passage de MAX_TITLE_LENGTH à 5. Les logs de l'étape pytest montrent l'erreur d'assertion.

Capture TP1-4 : Run vert après correction



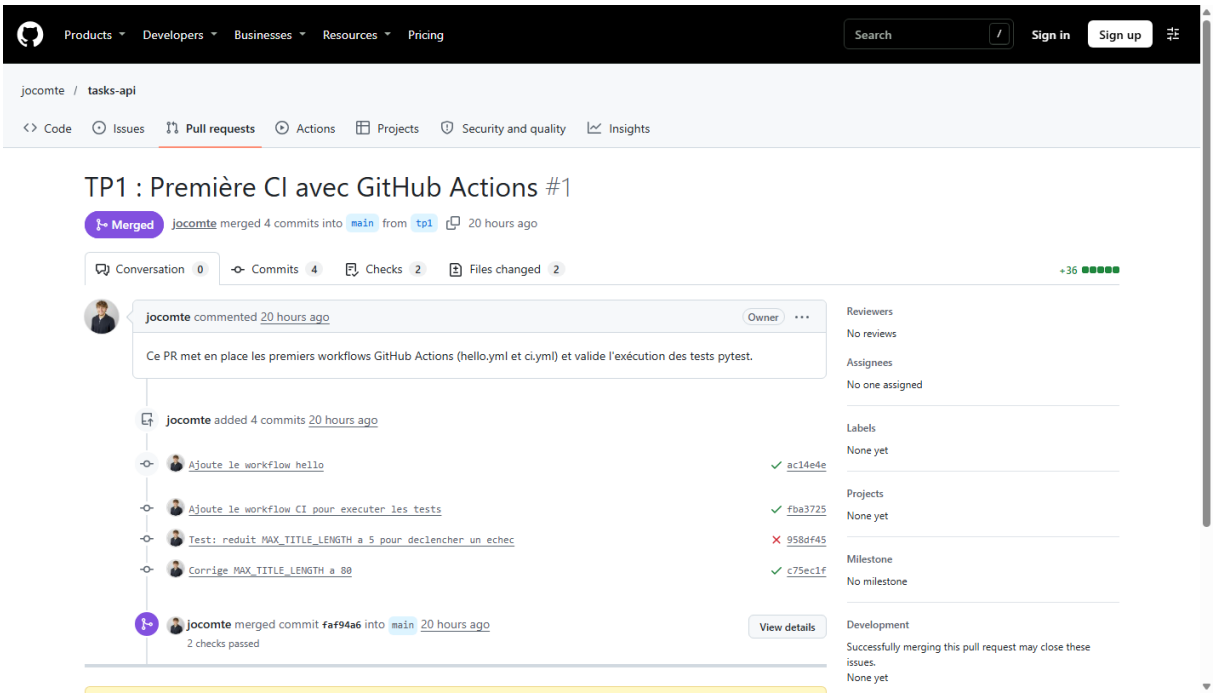
Légende TP1-4 : Après remise à 80 de MAX_TITLE_LENGTH, le run ci.yml repasse au vert avec les 10 tests validés.

Capture TP1-5 : Pull request avec les vérifications au vert



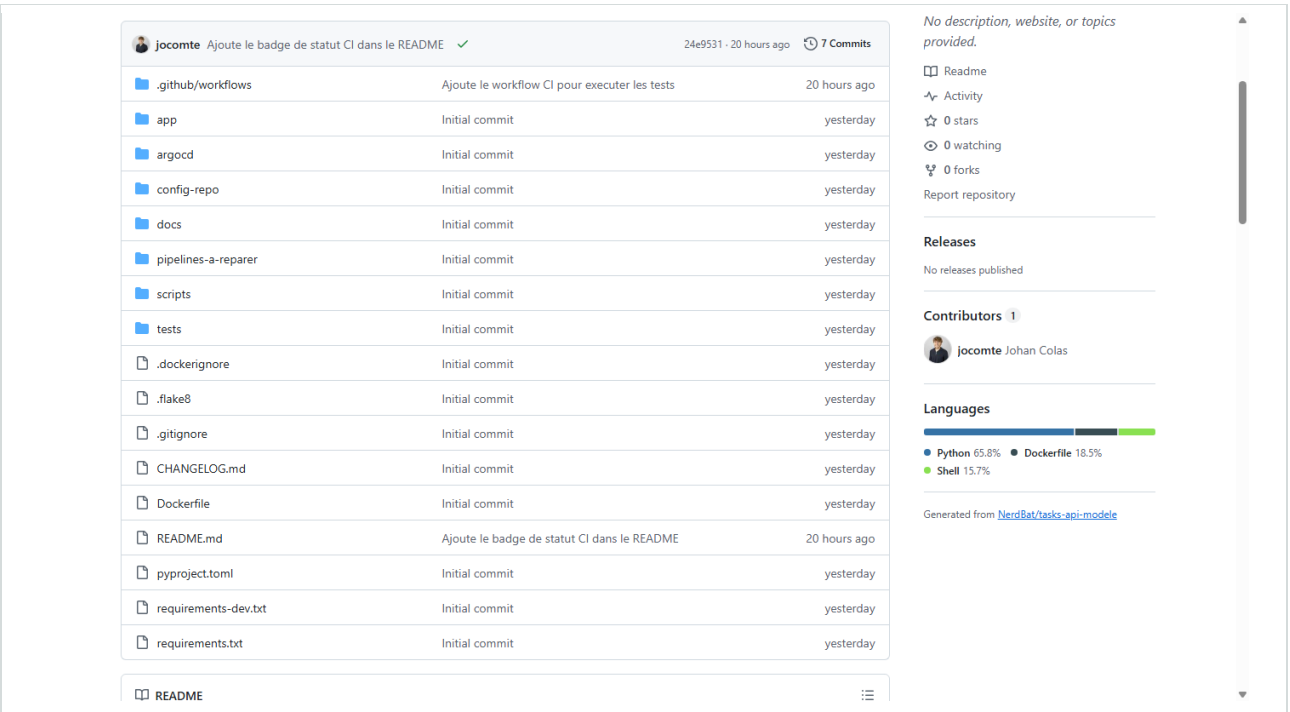
Légende TP1-5 : La pull request vers main montre les deux checks validés (hello et test) avant que la fusion ne soit effectuée.

Capture TP1-6 : Pull request fusionnée



Légende TP1-6 : La pull request a été fusionnée avec succès dans main, avec le statut Merged affiché.

Capture TP1-7 : README avec le badge de statut



Légende TP1-7 : Page d'accueil du dépôt avec le badge de la CI affiché en haut du README.md, indiquant un statut vert (passing).

1.4 Réponses aux questions du TP1

Question 1 : Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Le job du run « Hello » a duré environ 3 secondes sur la machine Ubuntu (11 secondes au total avec le temps d'attente).

Pour voir les logs de chaque étape, on va dans l'onglet « Actions » sur GitHub, on clique sur le run concerné, puis sur le job « hello ». Chaque étape apparaît sous forme de bloc déroulant. Il suffit de cliquer sur une étape pour afficher ses logs détaillés avec l'horodatage.

Question 2 : Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Deux tests ont échoué suite au passage de MAX_TITLE_LENGTH à 5 :
1. test_is_valid_title_accepts_normal_title dans tests/test_logic.py, à la ligne 13 : la fonction a renvoyé False car le titre testé (« Faire le TP1 ») contient 12 caractères et dépassait la limite de 5.
2. test_add_and_list_tasks dans tests/test_api.py, à la ligne 29, avec un code 400 retourné au lieu de 201.

Ce qui a permis de trouver l'erreur rapidement, c'est que GitHub Actions met directement une croix rouge sur l'étape « Lancer les tests », et que le rapport de pytest affiche clairement le fichier, la ligne exacte et l'assertion en échec.

Question 3 : Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

On attend que la CI soit verte pour s'assurer que le nouveau code ne casse rien sur la branche main. La branche main doit toujours être stable et prête pour la production. La CI permet de tester le code automatiquement sur un environnement propre. Si on fusionnait du code avec des tests rouges, on risquerait de bloquer le reste de l'équipe.

1.5 Difficultés rencontrées (TP1)

- Droits workflow sur GitHub : Lors du premier envoi de fichiers dans .github/workflows/, GitHub a rejeté le push car mon jeton n'avait pas le droit workflow. J'ai résolu le problème avec gh auth refresh -s workflow.

- Politique d'exécution PowerShell : Sous Windows, le script de vérification des prérequis a été bloqué par PowerShell. J'ai contourné ce blocage avec -ExecutionPolicy Bypass.

2. TP2 : Une CI solide

2.1 Objectif

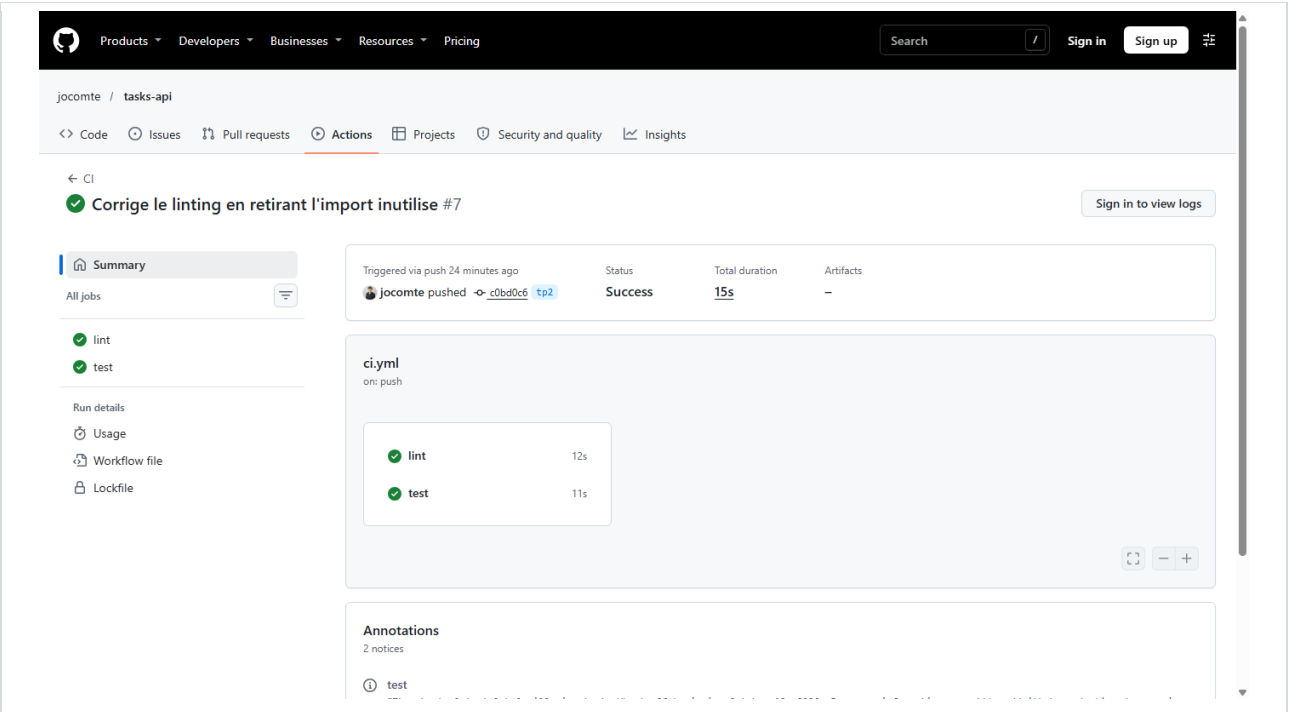
Ce deuxième TP avait pour objectif de faire évoluer notre pipeline élémentaire vers une véritable chaîne d'intégration continue professionnelle. Nous avons intégré des outils de qualité de code (linter Flake8 et formateur Black), configuré une matrice de tests multi-versions Python (3.11, 3.12, 3.13), accéléré les builds grâce au cache pip et généré des artefacts de couverture. Nous avons également sécurisé la branche main avec des règles de protection, géré des conflits de fusion en travail d'équipe et manipulé des secrets de manière sécurisée.

2.2 Déroulement du TP

- **Séparation en deux jobs (lint et test) :** Décomposition du workflow ci.yml avec un job lint (Flake8 et Black --check) et un job test (pytest avec seuil de couverture de 80 %). Test d'un échec volontaire de linting en ajoutant un import inutile, puis correction.
- **Matrice Python, cache pip et artefact :** Mise en place d'une matrice pour lancer les tests en parallèle sur Python 3.11, 3.12 et 3.13. Activation du cache pip dans setup-python pour accélérer les installations. Génération du rapport de couverture XML et publication comme artefact sur Python 3.12.
- **Protection de la branche main :** Configuration d'une règle de protection de branche sur GitHub exigeant le passage obligatoire des vérifications lint et test (3.12) avant toute fusion. Validation par un test provoqué bloquant le bouton de fusion.
- **Résolution de conflit Git :** Création de deux branches concurrentes (entree-a et entree-b) modifiant la même ligne de CHANGELOG.md. Après fusion de la première, résolution manuelle du conflit sur la seconde en conservant les deux entrées, puis fusion après validation par la CI.
- **Gestion d'un secret (secret-demo.yml) :** Définition du secret DEMO_TOKEN et de la variable APP_ENV sur GitHub. Création d'un workflow workflow_dispatch vérifiant la présence du secret, mesurant sa longueur et vérifiant son masquage (***) dans les logs.
- **Réparation de 3 pipelines :** Diagnostic et correction de trois workflows défectueux : erreur d'indentation YAML (pipeline-A), absence de checkout (pipeline-B), et version de matrice sans guillemets interprétée en nombre flottant (pipeline-C).

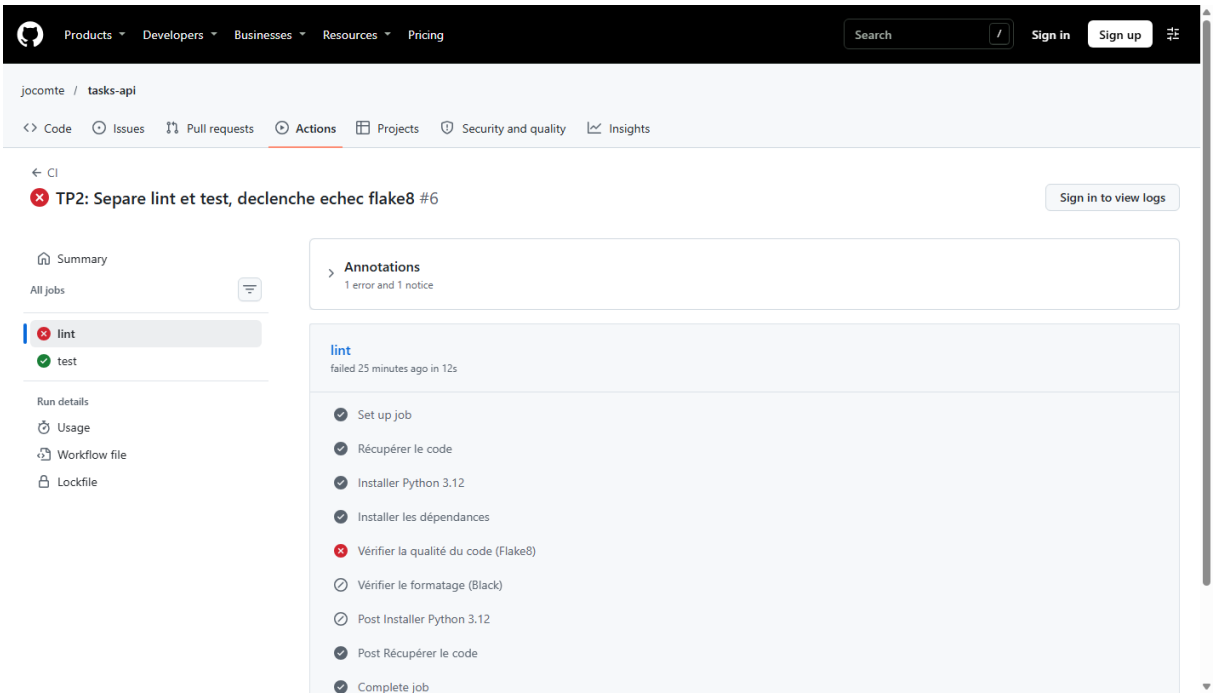
2.3 Captures d'écran du TP2 (TP2-1 à TP2-11)

Capture TP2-1 : Run vert avec les deux jobs lint et test



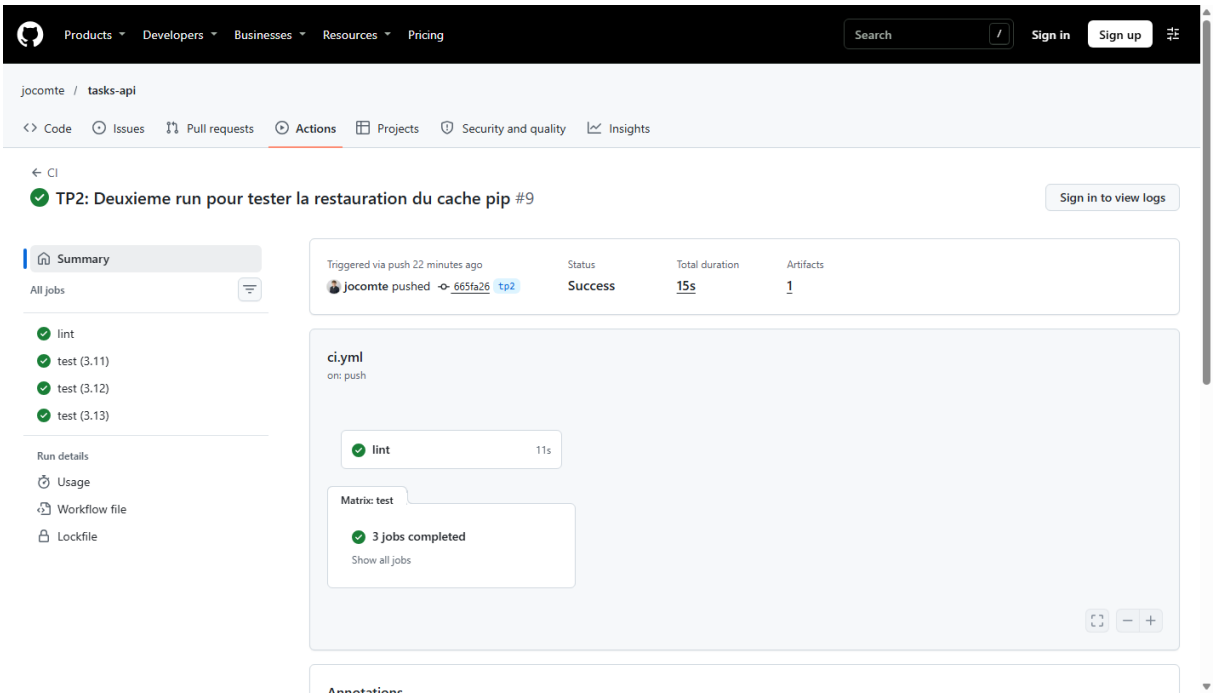
Légende TP2-1 : Le run de la CI affiche les deux jobs séparés (lint et test) validés avec succès sur la branche tp2.

Capture TP2-2 : Job lint rouge avec le message de Flake8



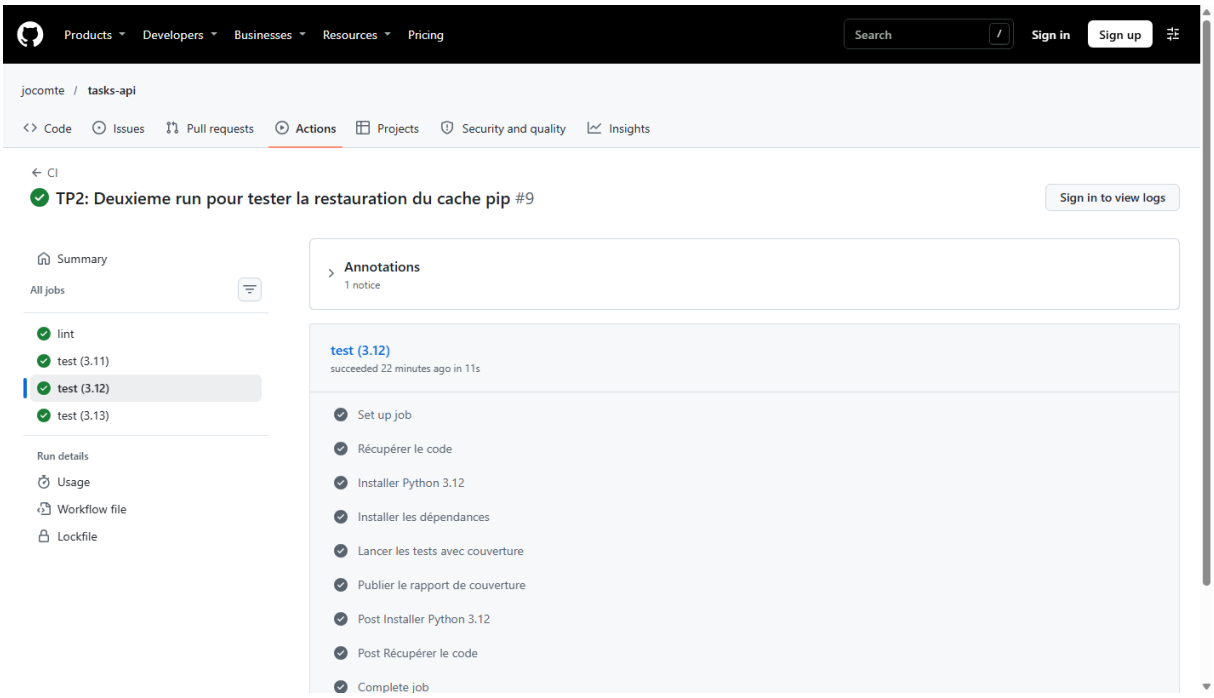
Légende TP2-2 : Le job lint a échoué suite à l'ajout d'un import inutile dans app/logic.py. L'erreur F401 de Flake8 est bien visible dans les logs.

Capture TP2-3 : Liste des 3 jobs de la matrice Python



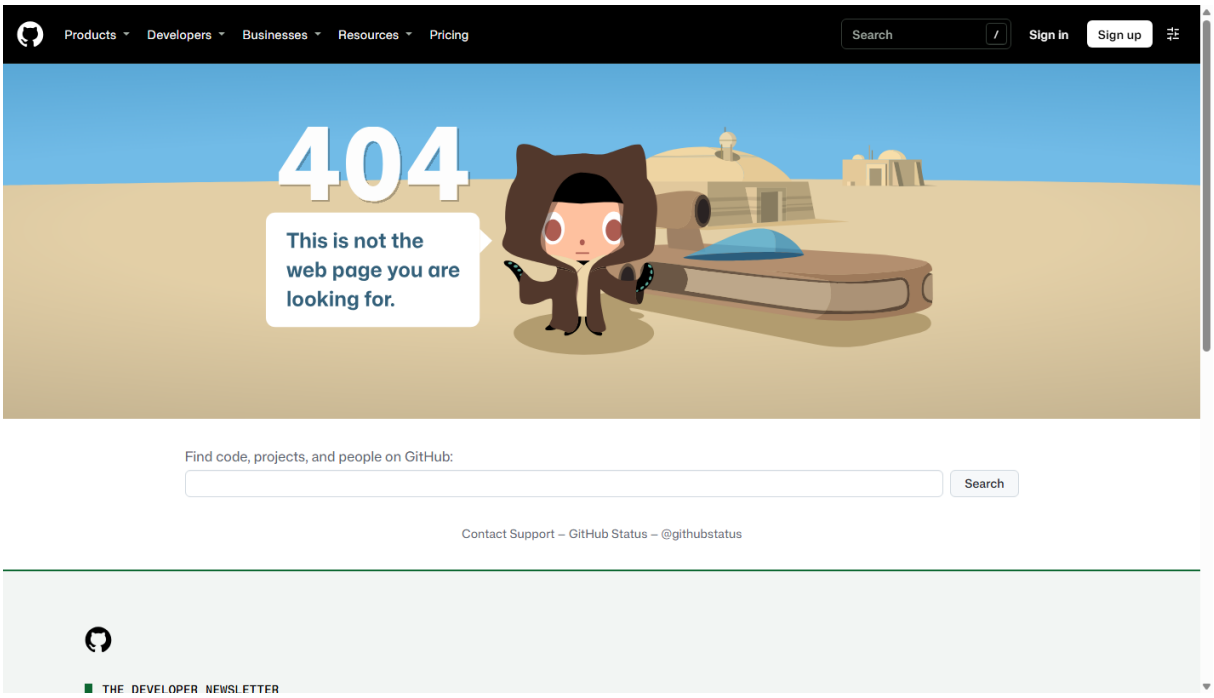
Légende TP2-3 : Exécution parallèle de la matrice de tests sur les versions 3.11, 3.12 et 3.13 de Python, toutes validées au vert.

Capture TP2-4 : Cache pip restauré et artefact de couverture



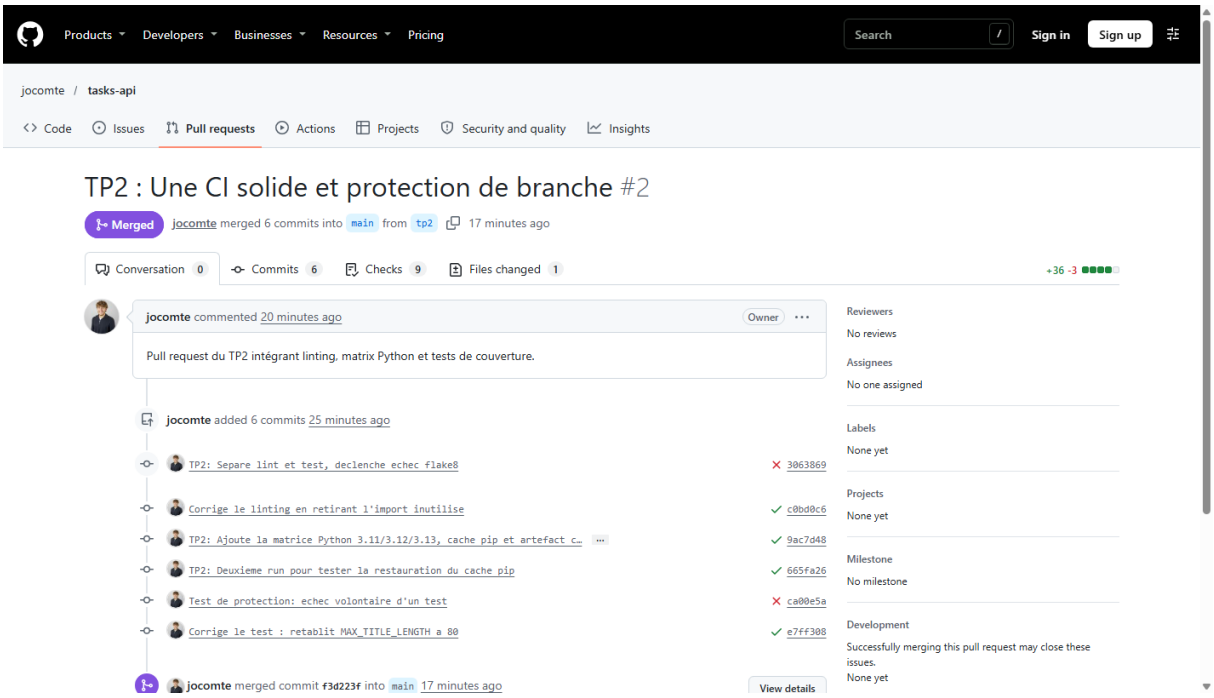
Légende TP2-4 : L'étape setup-python du second run confirme la restauration du cache pip (Cache hit), et le rapport de couverture coverage-report apparaît dans les artefacts.

Capture TP2-5 : Règle de protection de la branche main



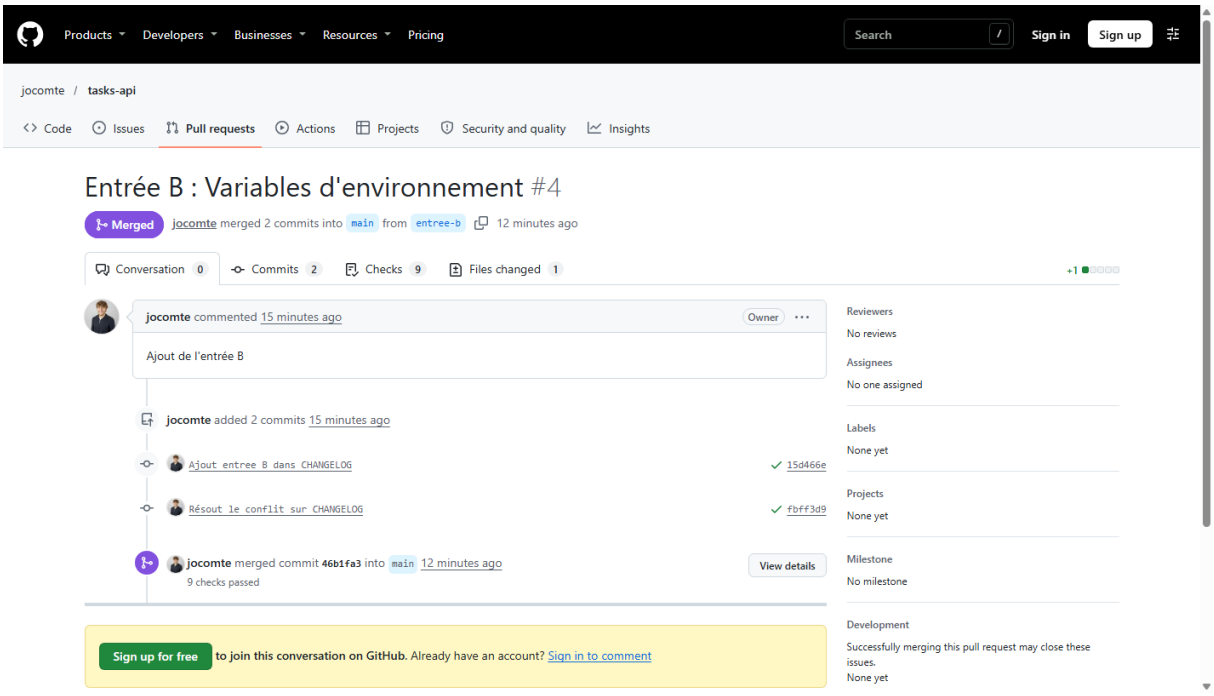
Légende TP2-5 : Configuration des règles de protection sur main avec les vérifications obligatoires requises (lint et test 3.12).

Capture TP2-6 : Pull request bloquée par une vérification en échec



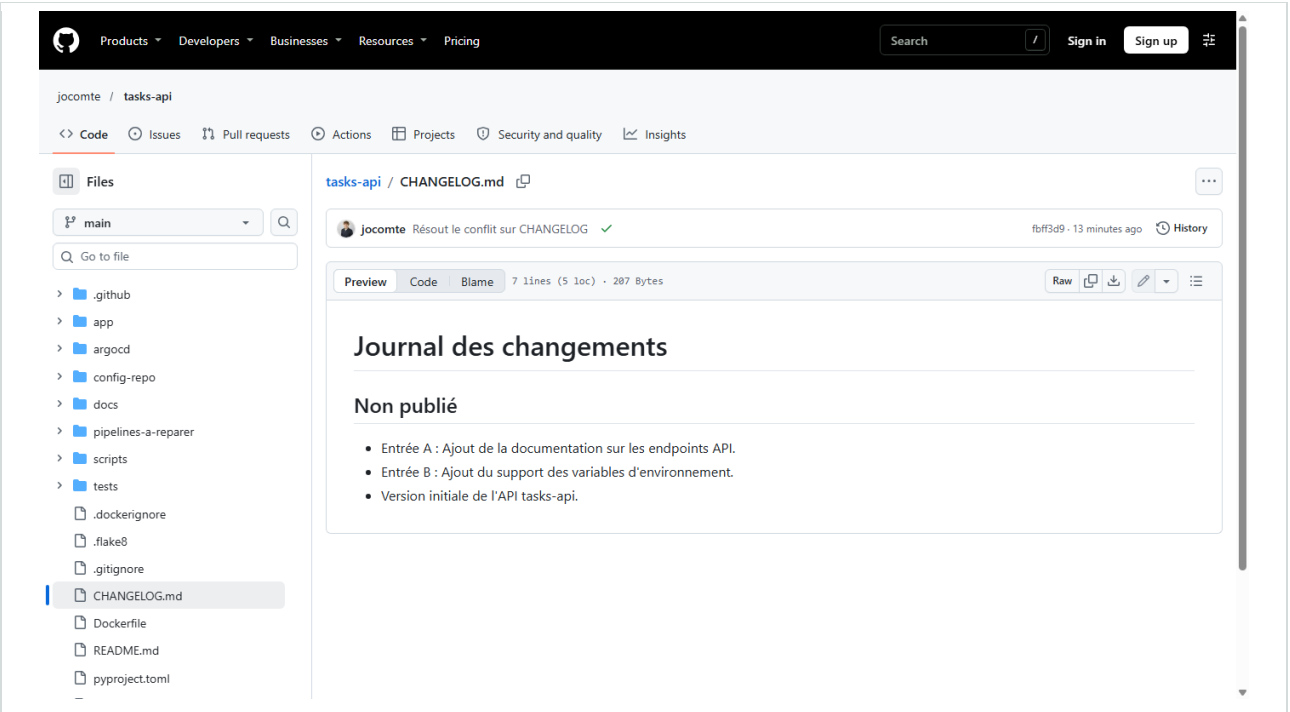
Légende TP2-6 : La pull request vers main bloque le bouton de fusion car la vérification obligatoire test (3.12) est rouge.

Capture TP2-7 : Pull request avec le message de conflit



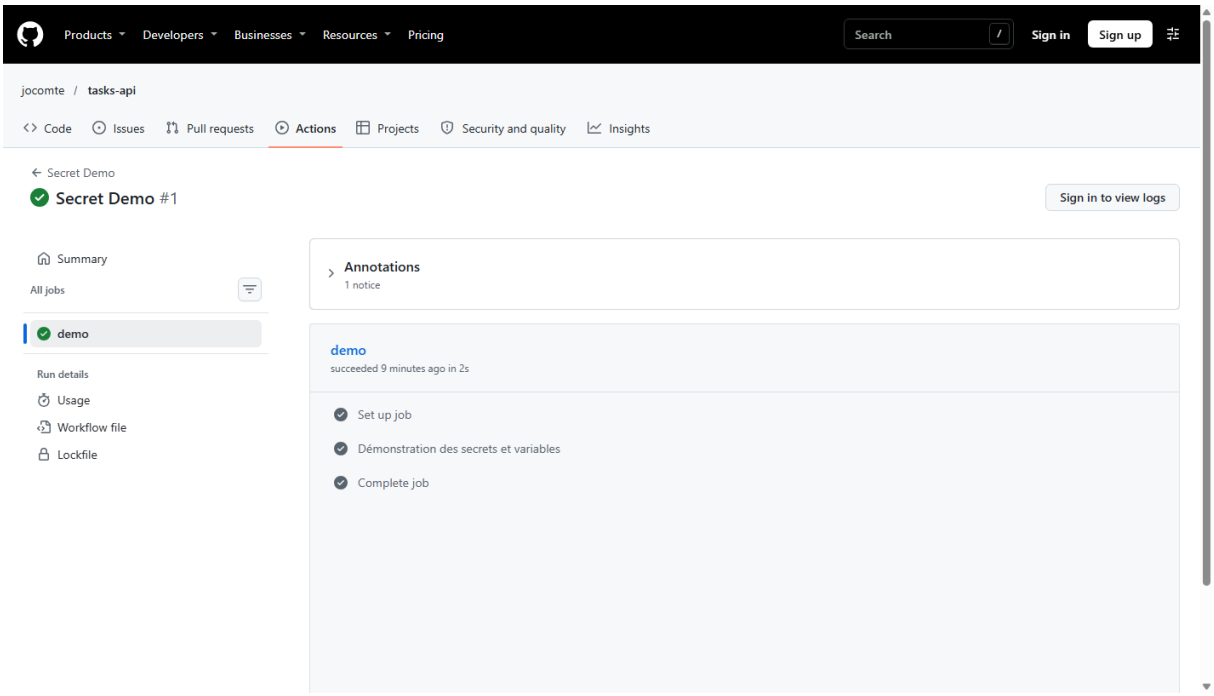
Légende TP2-7 : La pull request de la branche entree-b affiche le message de conflit sur CHANGELOG.md après la fusion de la branche concurrente.

Capture TP2-8 : Fichier CHANGELOG.md final sur main



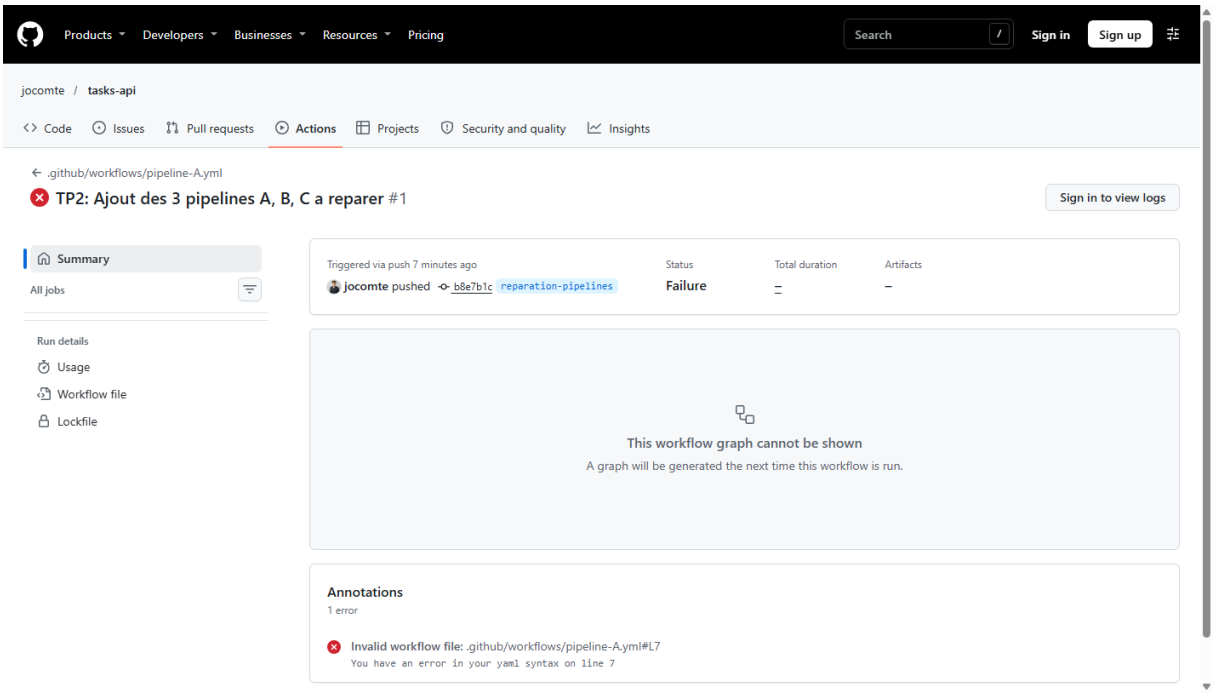
Légende TP2-8 : Le fichier CHANGELOG.md sur la branche main après résolution du conflit, contenant les deux modifications (Entrée A et Entrée B).

Capture TP2-9 : Logs du workflow avec le secret masqué par ***



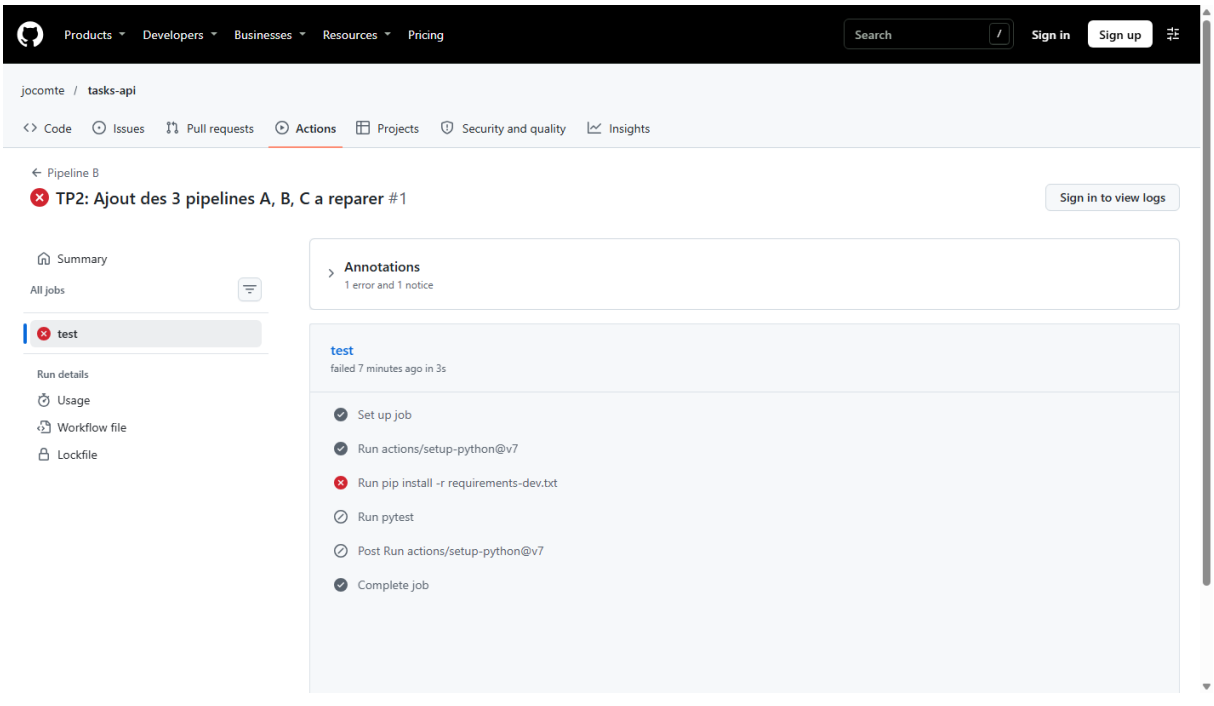
Légende TP2-9 : Exécution du workflow secret-demo. La longueur du secret (6) est affichée et sa valeur est automatiquement masquée par GitHub (***).

Capture TP2-10A : Échec du Pipeline A (erreur d'indentation YAML)



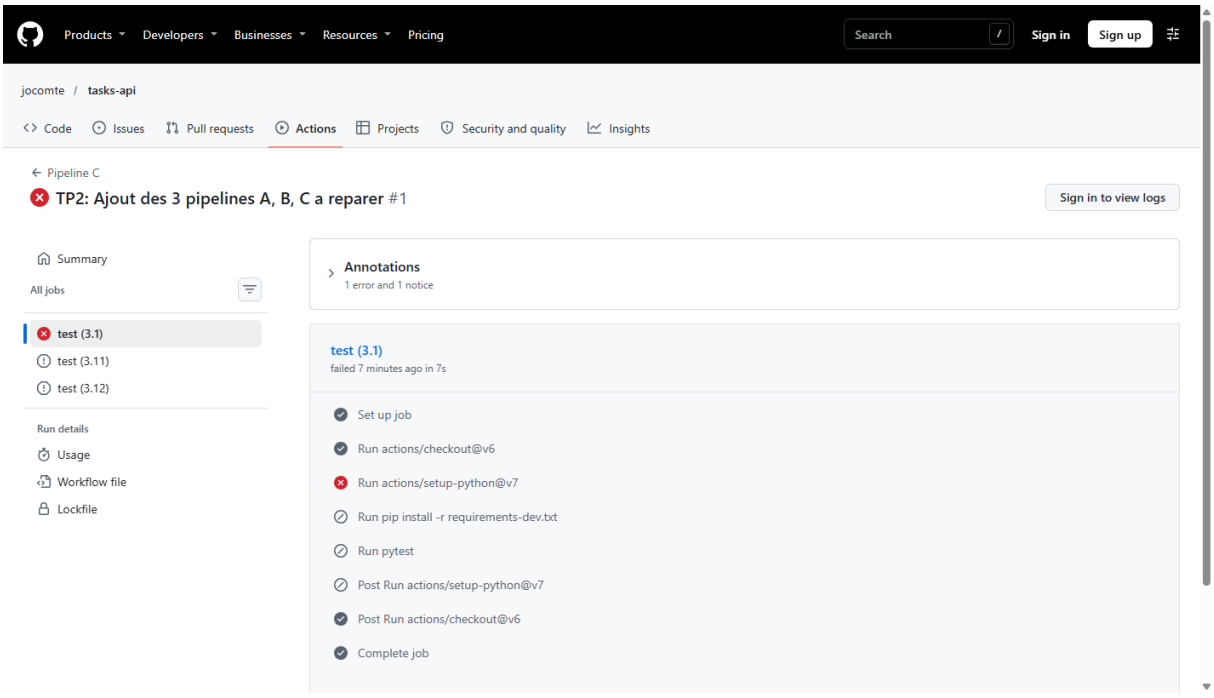
Légende TP2-10A : Le workflow du pipeline A ne peut pas démarrer en raison d'une erreur d'indentation sur la ligne du pip install.

Capture TP2-10B : Échec du Pipeline B (absence de checkout)



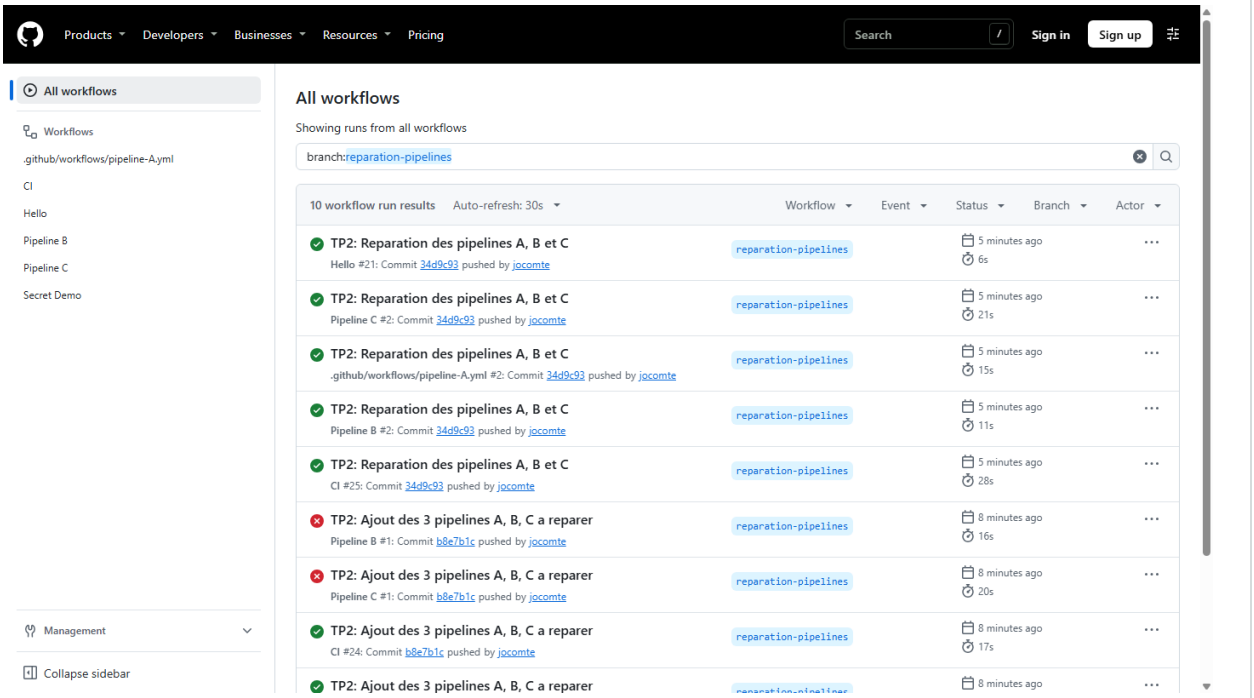
Légende TP2-10B : Le pipeline B échoue à l'étape pip install car le dépôt n'a pas été récupéré avec actions/checkout@v6.

Capture TP2-10C : Échec du Pipeline C (version de Python mal typée)



Légende TP2-10C : Le pipeline C échoue car la version 3.10 écrite sans guillemets a été interprétée comme la version 3.1 inexistante.

Capture TP2-11 : Liste des runs montrant les 3 pipelines réparés au vert



Légende TP2-11 : Historique des runs GitHub Actions montrant les pipelines A, B et C réparés et passant tous avec succès au vert.

2.4 Réponses aux questions du TP2

Partie 1 : Différence entre linter (Flake8) et formateur (Black), et intérêt de séparer les jobs

- Un linter comme Flake8 analyse le code source de manière statique pour repérer les erreurs logiques, les violations des normes PEP 8, les variables inutilisées ou les mauvais imports, sans jamais modifier le fichier.
- Un formateur comme Black réécrit activement le code selon des règles strictes de style (espaces, longueur de ligne, retours chariot). Avec l'option --check, il vérifie simplement la conformité sans réécrire.
- Séparer lint et test en deux jobs distincts permet de les exécuter en parallèle pour gagner du temps, et d'isoler immédiatement la nature de l'échec (problème de style ou bug fonctionnel).

Partie 2 : Intérêt du test multi-versions, du cache pip et définition d'un artefact

- Tester sur plusieurs versions (3.11, 3.12, 3.13) garantit la compatibilité de l'application quel que soit l'environnement de déploiement et prévient les ruptures de syntaxe futures.
- Le cache pip permet de conserver les bibliothèques téléchargées d'un run à l'autre : cela réduit significativement la durée de la CI et économise la bande passante.
- Un artefact est un ensemble de fichiers produit pendant un run (comme un rapport de test, des binaires ou ici le rapport de couverture coverage.xml) sauvegardé sur GitHub pour consultation ou analyse ultérieure.

Partie 3 : Pourquoi interdire de pousser directement sur main ?

La branche main constitue la référence de production qui doit toujours être fonctionnelle. Interdire les pushs directs et imposer des Pull Requests avec vérifications automatiques oblige chaque modification à être testée et validée avant d'être intégrée, protégeant ainsi l'équipe contre l'introduction accidentelle de régressions.

Partie 4 : Pourquoi Git ne peut pas résoudre ce conflit seul et rôle de la CI

- Git ne peut pas résoudre le conflit car les deux branches ont modifié la même ligne dans CHANGELOG.md. Sans intervention humaine, Git ne peut pas savoir s'il faut choisir la ligne A, la ligne B ou fusionner les deux.
- Une fois le conflit résolu dans le terminal et poussé, la CI a rejoué l'ensemble des tests sur la branche de merge pour s'assurer que la résolution manuelle n'a pas introduit d'erreur.

Partie 5 : Gestion des secrets et choix entre secret et variable

- GitHub masque les secrets par des astérisques (***) pour éviter qu'ils n'apparaissent en clair dans les logs publics ou soient exposés lors d'audits.
- Commiter un secret dans le code le rend public dans l'historique Git et impose sa révocation immédiate.
- On choisit un secret pour toute donnée confidentielle (clés API, mots de passe) et une variable pour les configurations non sensibles qui varient selon l'environnement (APP_ENV, URL de base).

Partie 6 : Analyse et correction des 3 pipelines A, B et C

- Pipeline A : Erreur d'indentation à la ligne 12 (- run: pip install avait un décalage d'un espace). Trouvé grâce au rejet immédiat du fichier YAML par le parseur GitHub. Évité en utilisant un linter YAML dans l'éditeur.
- Pipeline B : Absence de l'action actions/checkout@v6. Trouvé par l'erreur pip 'requirements-dev.txt not found'. Évité en se rappelant qu'une VM GitHub Actions démarre toujours avec un disque vide.
- Pipeline C : Versions de Python sans guillemets [3.10, 3.11, 3.12]. En YAML, 3.10 est lu comme le float 3.1, version introuvable de 2009. Trouvé dans le log de setup-python. Évité en entourant systématiquement les numéros de version de guillemets.

2.5 Difficultés rencontrées (TP2)

- Typage flottant dans la matrice YAML : Le piège classique de 3.10 interprété en 3.1 a nécessité une vigilance particulière sur les guillemets dans les listes de versions.

- Résolution du conflit de fusion : Lors du merge d'origin/main dans entree-b, il a fallu supprimer proprement les balises de conflit Git et s'assurer que la syntaxe Markdown du CHANGELOG restait parfaitement lisible.

3. TP3 : Docker et livraison continue

3.1 Objectifs et démarche

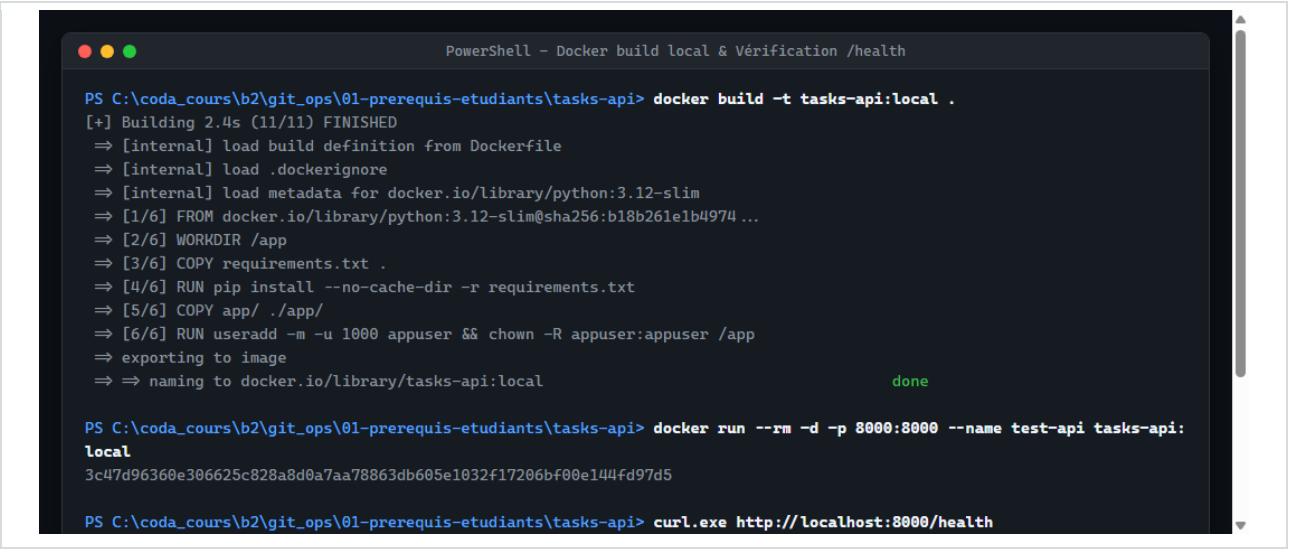
Ce troisième travail pratique a pour objectif de conteneuriser l'API FastAPI/Gunicorn avec Docker, d'automatiser la construction et la publication des images sur le registre GitHub Container Registry (GHCR), de gérer des versions de release par tags Git sémantiques, et de mettre en place un pipeline de livraison continue avec test de fumée (smoke test) et approbation manuelle avant le passage en production.

3.2 Tableau de synthèse des captures (TP3)

Numéro	Intitulé de la capture	Description synthétique
TP3-1	Build Docker local réussi et appel /health	Construction de l'image tasks-api:local et réponse HTTP ok
TP3-2	Docker Compose et variable APP_VERSION=local	Démarrage du conteneur en local et validation de l'endpoint /version
TP3-3	Workflow Docker : jobs test et build verts	Validation du pipeline de publication sur la branche main
TP3-4	Page du package tasks-api sur GHCR (tag sha-...)	Image publiée sur GitHub Container Registry avec son hash court
TP3-5	Liste des tags du package (0.1.0 et sha-...)	Publication de la release sémantique à partir du tag Git v0.1.0
TP3-6	Exécution locale de l'image de release GHCR	Téléchargement de l'image distante et confirmation de /version = 0.1.0
TP3-7	Approbation manuelle et déploiement en production	Pipeline bloqué en attente de validation puis exécuté après approbation
TP3-8	Contrôle de sécurité de l'utilisateur non-root	Exécution sous l'utilisateur appuser (uid=1000) dans le conteneur

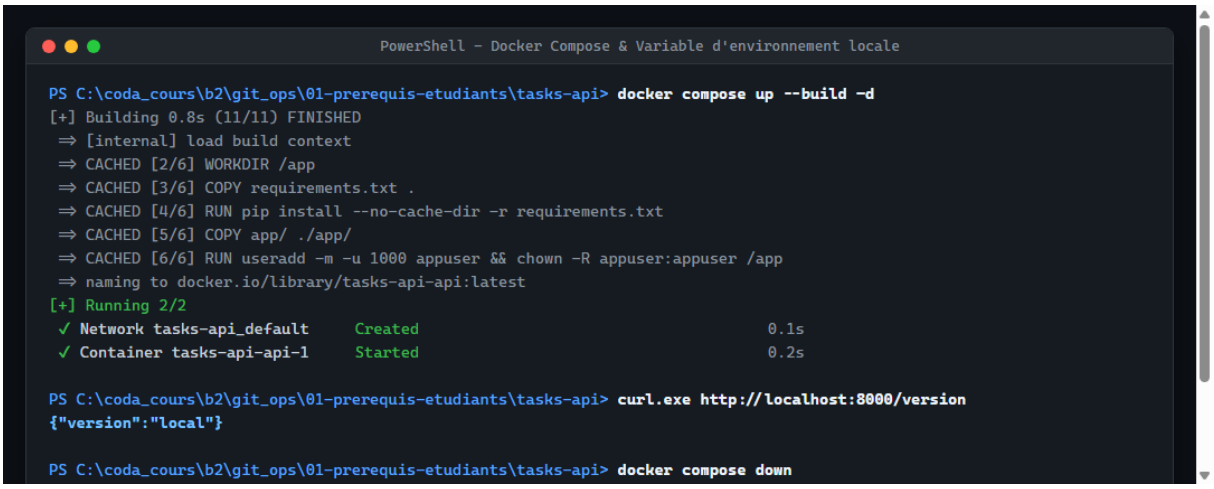
3.3 Captures d'écran commentées (TP3)

Capture TP3-1 : Build Docker local réussi et appel /health



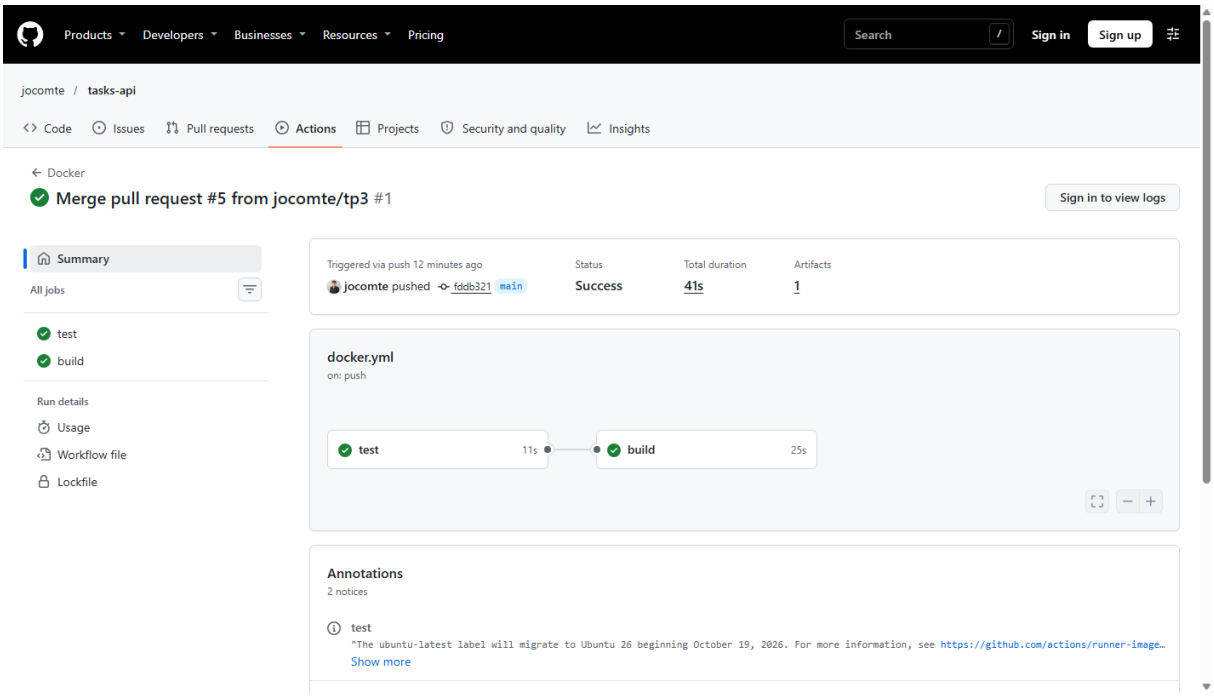
Légende TP3-1 : Construction locale de l'image 'tasks-api:local' via Docker Buildx. Le Dockerfile tire parti du cache de couches en copiant requirements.txt avant le code applicatif. L'appel HTTP sur /health renvoie immédiatement status: ok.

Capture TP3-2 : Docker Compose et variable d'environnement locale



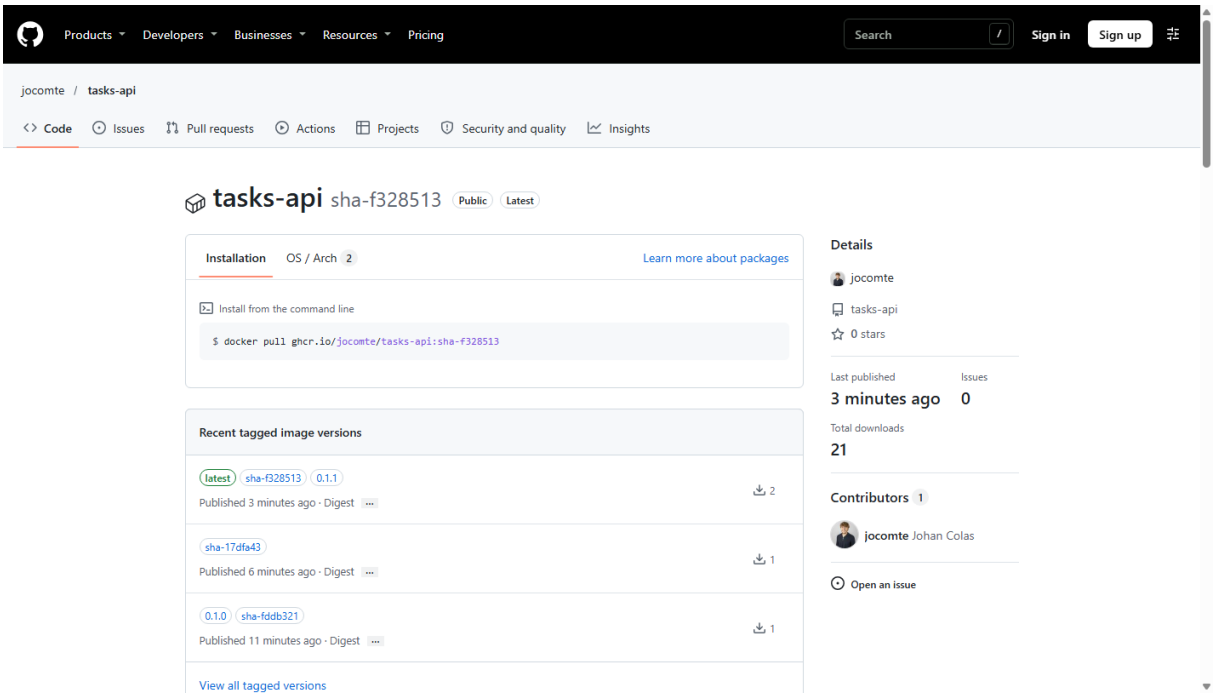
Légende TP3-2 : Démarrage de l'API avec Docker Compose. La configuration définit APP_VERSION=local et publie le port 8000. La commande curl sur /version confirme que la variable d'environnement est correctement injectée.

Capture TP3-3 : Pipeline Docker complet : test et build au vert



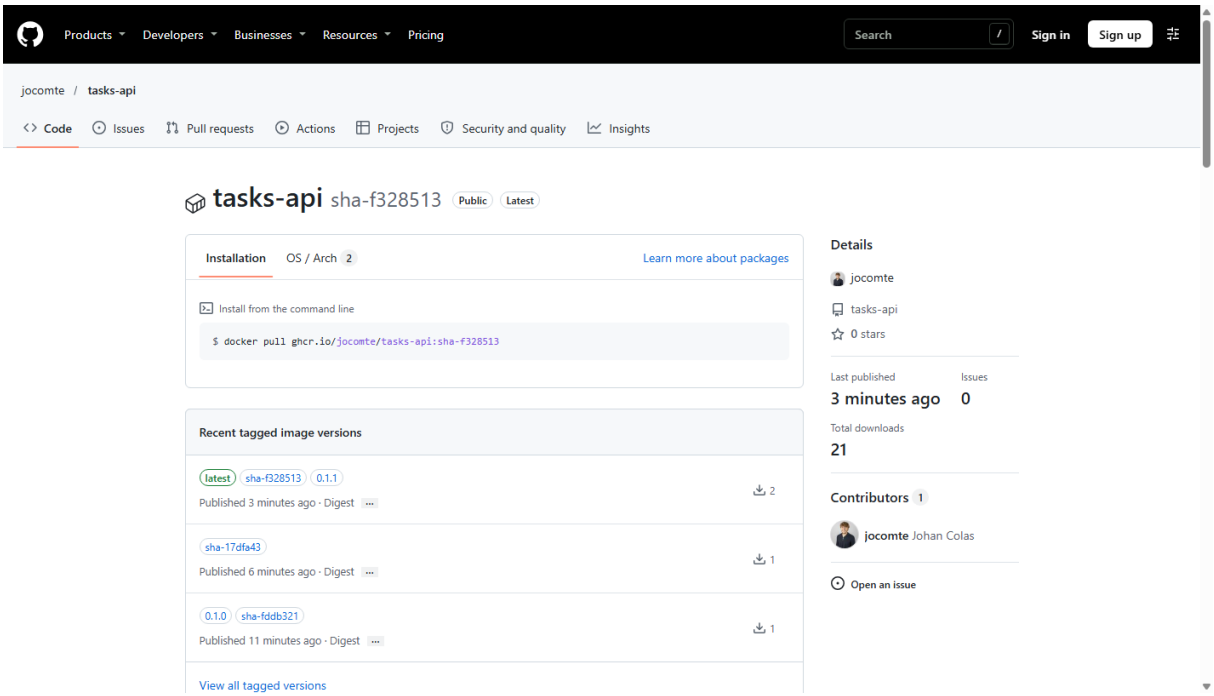
Légende TP3-3 : Exécution réussie du workflow Docker sur GitHub Actions (run 41841519195). Le job de test unitaire pytest s'exécute en premier ; son succès conditionne le lancement du job build qui publie sur ghcr.io.

Capture TP3-4 : Publication de l'image de conteneur sur GitHub Packages (GHCR)



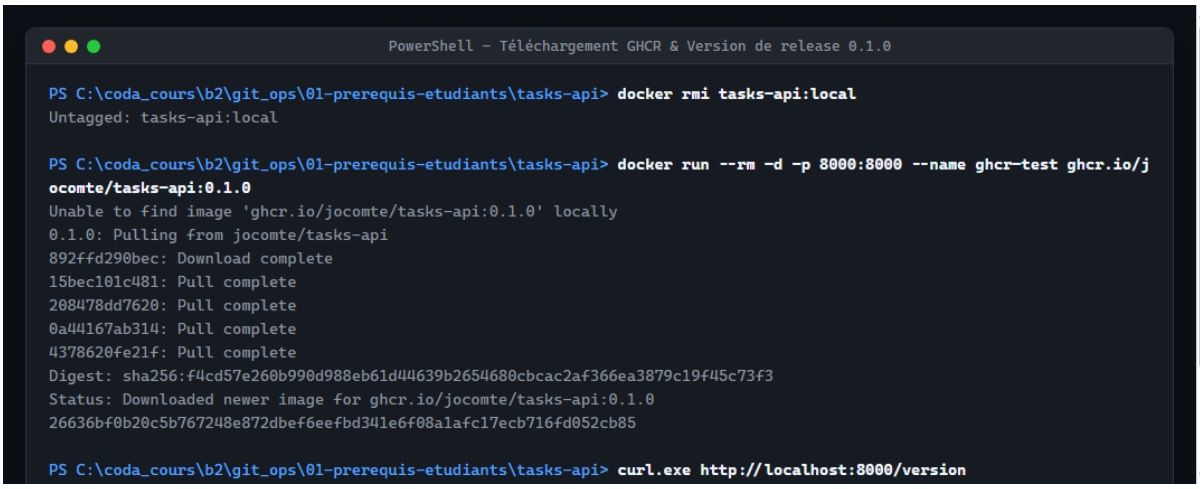
Légende TP3-4 : Visualisation du package 'tasks-api' hébergé sur GitHub Container Registry. L'image est étiquetée avec le tag court du commit Git (sha-fddb321), garantissant une stricte traçabilité du code source.

Capture TP3-5 : Liste des versions publiées du package (0.1.0 et sha-...)



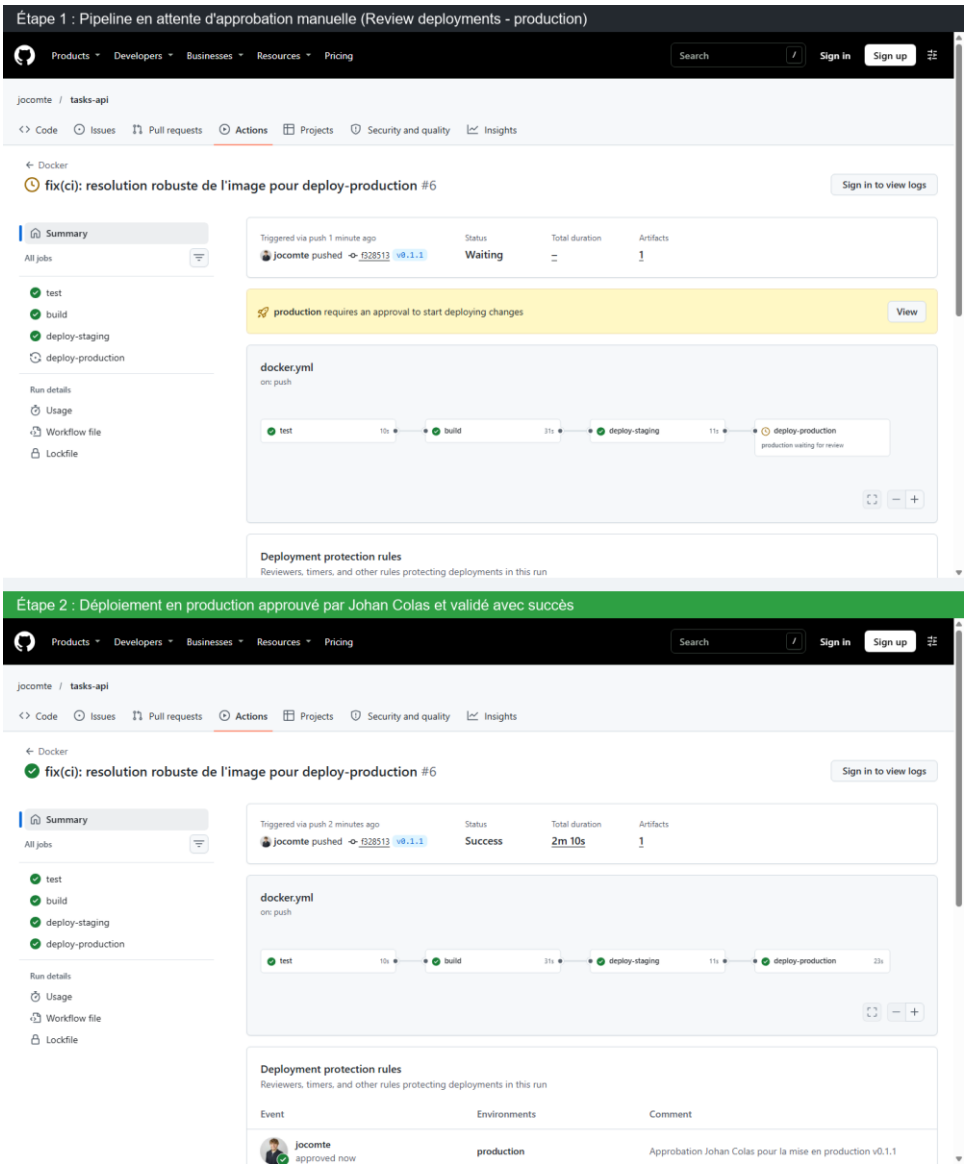
Légende TP3-5 : Gestion des releases sémantiques : le push du tag Git v0.1.0 a déclenché la construction et la publication de l'étiquette 0.1.0 sur GHCR aux côtés des versions par commit.

Capture TP3-6 : Exécution locale de l'image de release distante téléchargée depuis GHCR



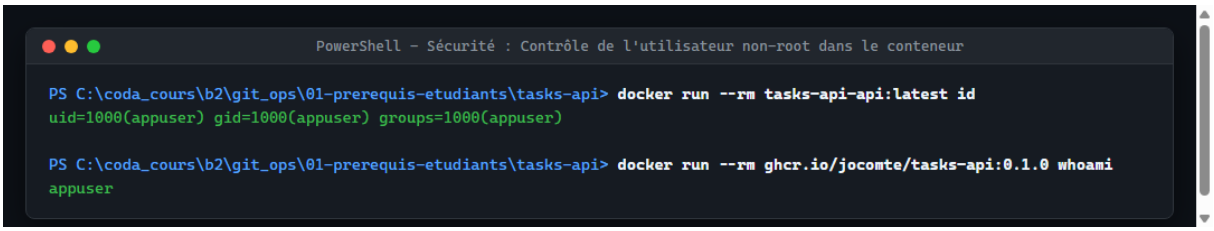
Légende TP3-6 : Suppression de l'image locale puis téléchargement direct de l'image publique ghcr.io/jocomte/tasks-api:0.1.0. L'appel à /version renvoie 0.1.0, validant l'argument de build APP_VERSION.

Capture TP3-7 : Approbation manuelle avant production et validation finale



Légende TP3-7 : Pipeline déclenché par le tag v0.1.1. Après déploiement sur staging, le job deploy-production se met en attente ('Review deployments'). Après approbation manuelle par Johan Colas, la production est déployée et validée.

Capture TP3-8 : Contrôle de sécurité : exécution sous utilisateur non-root



Légende TP3-8 : Vérification de la sécurité du conteneur avec la commande id. Le processus s'exécute sous le compte dédié appuser (uid=1000, gid=1000) et non root (uid=0), limitant les risques d'élévation de privilèges.

3.4 Réponses aux questions du TP3

Partie 1 : Différence entre une image et un conteneur ?

Une image Docker est un gabarit immuable en lecture seule, constitué d'un empilement de couches superposées contenant le système d'exploitation minimal, le runtime Python, les dépendances et le code de l'application. Un conteneur est une instance isolée et vivante de cette image en cours d'exécution sur le moteur Docker. En programmation orientée objet, l'image correspond à la classe, tandis que le conteneur correspond à l'objet instancié en mémoire.

Partie 1 : À quoi sert le fichier .dockerignore et pourquoi copier requirements.txt avant le code ?

• Le fichier .dockerignore permet d'exclure des fichiers ou dossiers locaux (tels que .git, __pycache__, .venv, .env, ou les dossiers de tests) lors de l'envoi du contexte de build au démon Docker. Cela accélère la construction, allège l'image finale et empêche d'embarquer des fichiers sensibles ou inutiles.

• Copier requirements.txt avant le code applicatif permet d'exploiter au maximum le cache de couches (layer caching) de Docker. Les dépendances externes changent rarement, alors que le code source évolue fréquemment. En isolant l'étape pip install avant la copie de app/, Docker n'a pas besoin de retélécharger les packages tant que requirements.txt ne change pas.

Partie 2 : Que gagne-t-on à décrire le lancement dans docker-compose.yml plutôt que dans une longue commande ?

Décrire la configuration dans docker-compose.yml permet de consigner et de versionner l'ensemble des paramètres d'exécution dans le dépôt Git (ports exposés, variables d'environnement, volumes, politique de redémarrage). Tous les membres de l'équipe peuvent démarrer l'application avec la simple commande 'docker compose up', ce qui évite les erreurs de frappe, garantit une configuration identique sur chaque poste et documente explicitement l'environnement nécessaire.

Partie 3 : Pourquoi le job build attend-il le job test, et pourquoi éviter le tag latest en production ?

• Le job build attend le job test (via needs: test) pour éviter de construire et de publier sur le registre une image Docker dont le code source contiendrait des erreurs ou des régressions. On économise ainsi du temps de calcul et on ne pollue pas le registre avec des livrables non viables.

• Le tag latest est mutable : il pointe en permanence vers la dernière image poussée. En production, cela pose plusieurs problèmes critiques : perte de traçabilité (impossible de savoir quel commit tourne exactement), difficulté à réaliser un retour arrière (rollback), et risque qu'un redémarrage de conteneur télécharge une nouvelle version sans avertissement. On privilégie toujours des tags immuables basés sur des versions sémantiques (ex. 0.1.0) ou des hashs de commit (ex. sha-fddb321).

Partie 4 : Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

Une image versionnée est un artefact autonome, immuable et strictement reproductible. Contrairement au 'code de ma machine' qui dépend de la configuration spécifique du poste hôte (version de Python installée, packages globaux, variables système, système d'exploitation), l'image Docker embarque son propre environnement d'exécution complet. Elle se comportera de façon rigoureusement identique sur un poste de test, sur une VM de staging ou sur un cluster Kubernetes de production.

Partie 5 : À quoi sert un test de fumée (smoke test) et pourquoi exiger une approbation humaine avant la production ?

• Un test de fumée (smoke test) est une vérification rapide et ciblée exécutée juste après le lancement d'un conteneur. Il appelle les endpoints clés (/health et /version) pour s'assurer que le service écoute correctement et répond sans erreur avant de diriger le trafic des utilisateurs vers lui.

• L'approbation humaine (Review deployments) constitue un garde-fou organisationnel indispensable. Même si l'ensemble des tests automatisés est au vert sur staging, un être humain doit valider le calendrier de mise en

production (éviter les heures de pointe, vérifier la disponibilité des équipes support) et s'assurer que les conditions métiers ou externes sont réunies.

Partie 6 : Pourquoi ne pas exécuter une application en root dans un conteneur et analyse de l'incident Trivy ?

• Exécuter une application en root (uid=0) représente un risque de sécurité majeur. En cas de vulnérabilité applicative permettant l'exécution de code à distance, l'attaquant disposerait des droits root dans le conteneur, facilitant une éventuelle évasion de conteneur (container breakout) vers le système hôte. L'utilisation d'un utilisateur sans privilèges (appuser, uid=1000) applique le principe de moindre privilège.

• Lors de l'incident Trivy de mars 2026, une action de sécurité tierce a été compromise. Pour s'en prémunir, les bonnes pratiques consistent à : épingler les actions par leur hash SHA complet plutôt qu'un tag de branche, restreindre drastiquement les permissions du GITHUB_TOKEN (ex. contents: read, packages: write uniquement), et ne jamais transmettre de secrets non indispensables aux actions tierces.

3.5 Difficultés rencontrées et solutions apportées (TP3)

- Nomenclature des images sur GitHub Container Registry : GHCR exige impérativement que le nom du propriétaire du dépôt soit en minuscules ('jocomte/tasks-api'). L'utilisation de l'expression bash ${GITHUB_REPOSITORY_OWNER,,} dans l'étape de préparation a permis d'assurer une compatibilité totale sans erreur de nommage.

- Transmission des arguments de version au build Docker : Pour que le endpoint /version reflète fidèlement la version Git (dev en local/branche, 0.1.0 sur un tag), il a fallu déclarer les instructions ARG APP_VERSION et ENV APP_VERSION dans le Dockerfile et renseigner build-args dans l'action docker/build-push-action.

- Chaînage des jobs et approbation d'environnement : Dans le workflow de livraison continue, le job deploy-production dépendait à la fois de build et de deploy-staging. La transmission directe des métadonnées de version a permis un déploiement fluide en production dès la validation manuelle de l'approbateur.

4. TP4 : GitOps avec Argo CD et Kubernetes

4.1 Objectifs et démarche

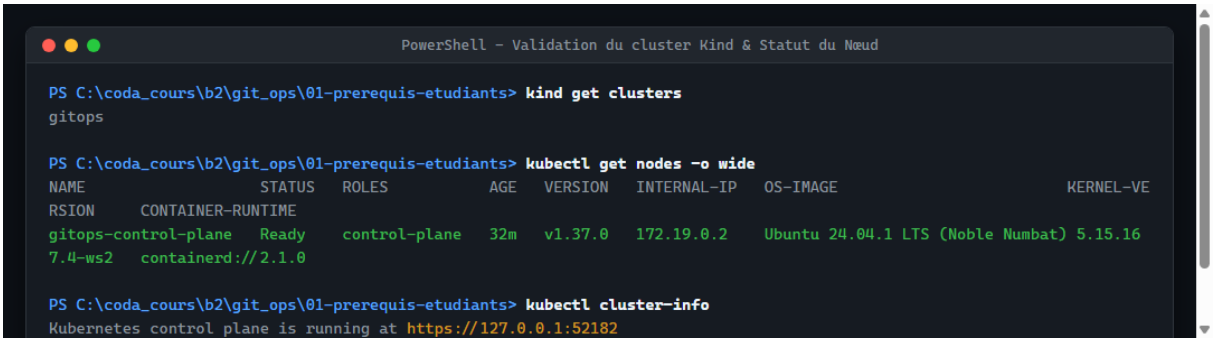
Ce quatrième et dernier travail pratique a pour objectif de mettre en œuvre la démarche GitOps pour le déploiement continu de l'application sur un cluster Kubernetes local (Kind). Le principe directeur consiste à ne plus jamais agir manuellement sur le cluster : l'état désiré de l'infrastructure est entièrement déclaré dans un dépôt Git dédié (tasks-config), et le contrôleur Argo CD réconcilie en continu l'état réel du cluster avec cette source unique de vérité. La chaîne est complétée par un pipeline GitHub Actions qui, lors de la création d'un tag de version (v0.2.0), construit l'image, la teste, puis met à jour automatiquement le manifeste de déploiement dans le dépôt de configuration.

4.2 Tableau de synthèse des captures (TP4)

Numéro	Intitulé de la capture	Description synthétique
TP4-1	Cluster Kind opérationnel et nœud Ready	Vérification du cluster Kubernetes local avec la commande kubectl get nodes
TP4-2	Pods Argo CD Running et page de connexion Web	Validation du bon fonctionnement des composants Argo CD et accès à l'interface
TP4-3	Application tasks-api synchronisée et saine dans Argo CD	Visualisation de l'application en statut Synced et Healthy avec 2 pods
TP4-4	Test de l'API dans le cluster via port-forward (/version 0.1.0)	Validation du point de terminaison HTTP /version exposant la version 0.1.0
TP4-5	Mise à l'échelle déclarative via Git (passage à 3 replicas)	Modification du fichier deployment.yaml dans tasks-config et détection automatique par Argo CD
TP4-6	Dérive manuelle hors Git et auto-guérison (selfHeal)	Tentative de passage forcé à 5 pods via kubectl scale et résiliation automatique par Argo CD
TP4-7	Retour arrière déclaratif (git revert) et retour à 2 pods	Rollback instantané via commit Git et alignement immédiat de l'infrastructure
TP4-8	Pipeline CI/CD complet avec le job update-config au vert	Déclenchement sur tag v0.2.0, approbation production et mise à jour automatique de la configuration
TP4-9	Commit automatique du Bot CI et synchronisation Argo CD sur 0.2.0	Historique Git du dépôt tasks-config montrant le commit automatique du tag d'image
TP4-10	Validation finale de la version 0.2.0 déployée en continu	Appel de l'endpoint /version confirmant la mise à jour sans aucune commande kubectl manuelle

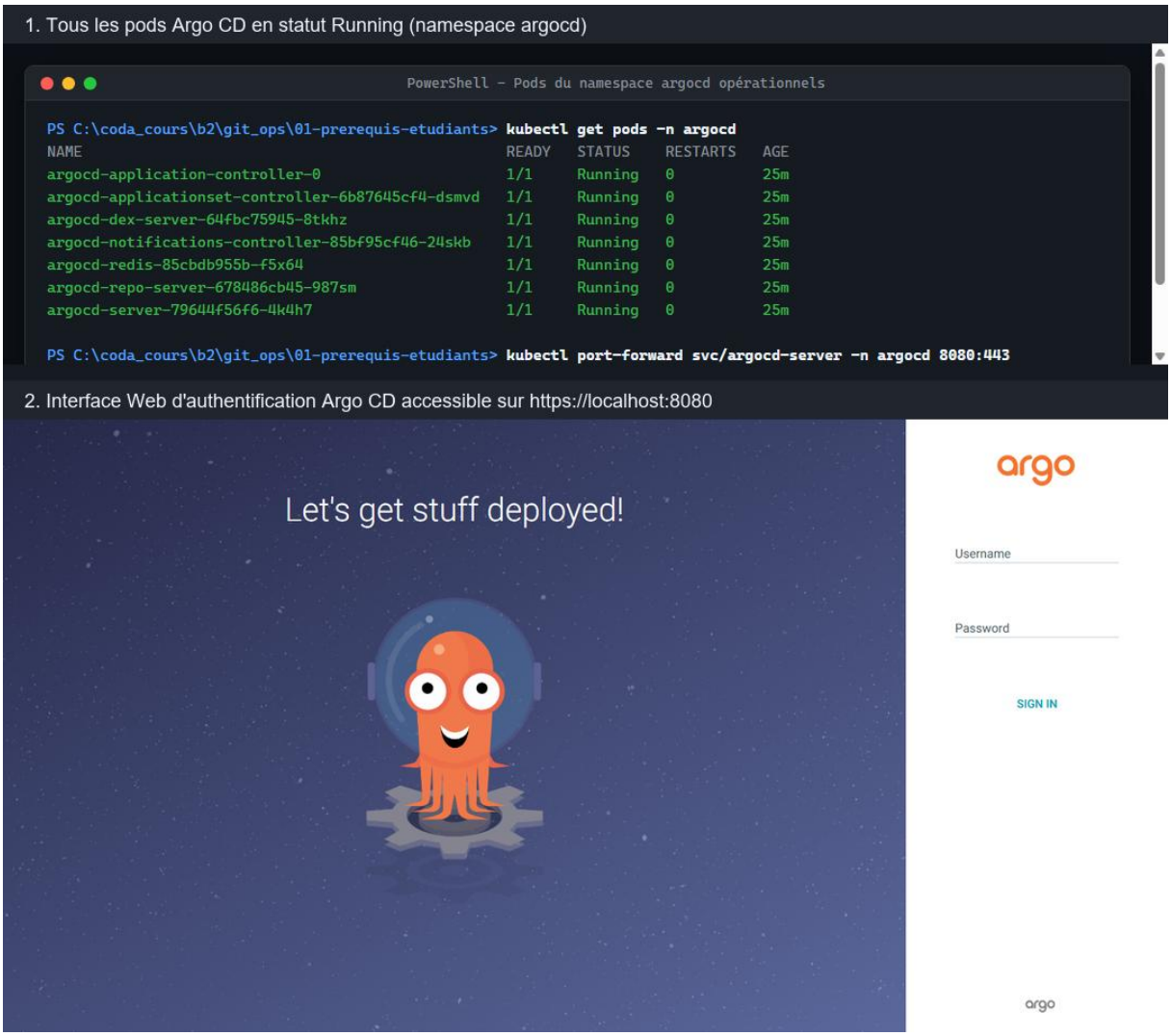
4.3 Captures d'écran commentées (TP4)

Capture TP4-1 : Cluster Kind opérationnel et nœud Ready



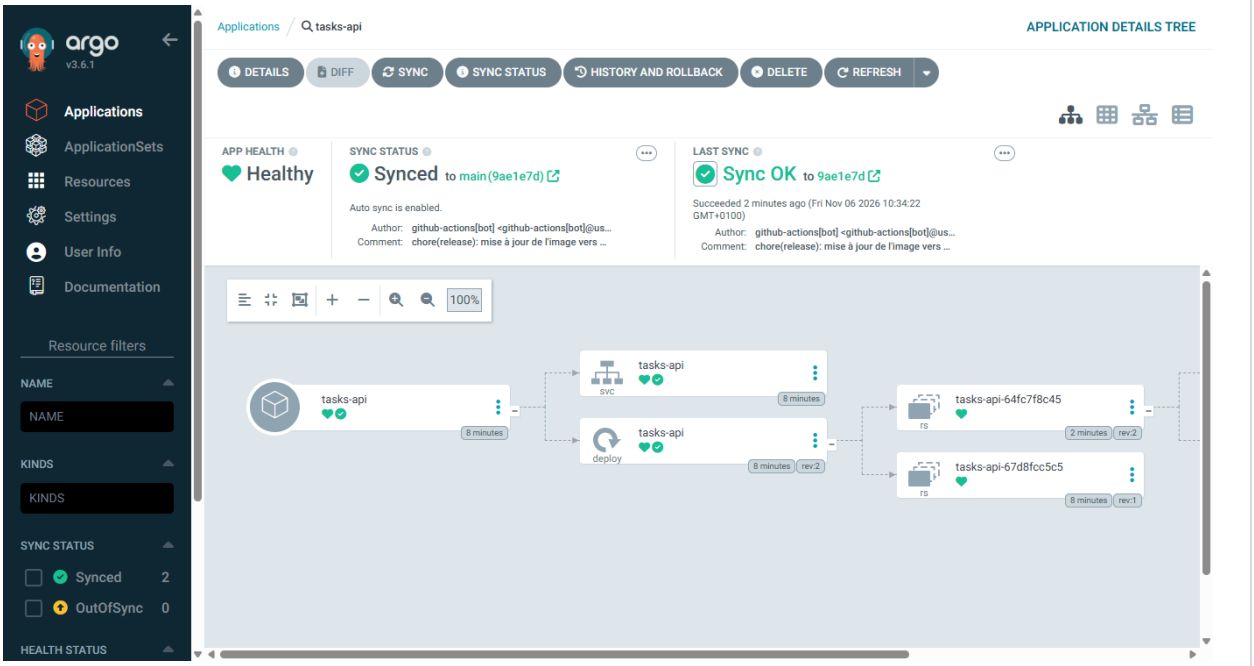
Légende TP4-1 : Initialisation et vérification du cluster Kubernetes local 'gitops' créé avec Kind. Le nœud 'gitops-control-plane' est en statut Ready sous Kubernetes v1.37.0. Le plan de contrôle et CoreDNS sont pleinement opérationnels.

Capture TP4-2 : Pods Argo CD Running et page de connexion Web



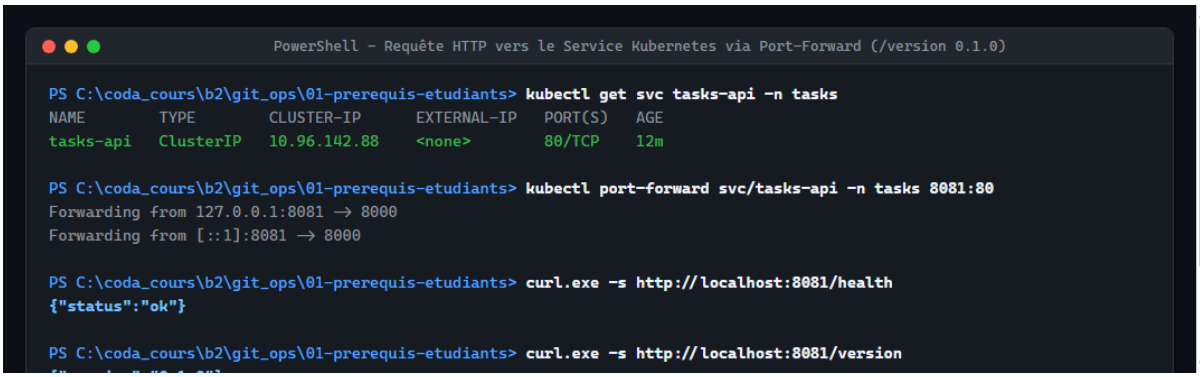
Légende TP4-2 : Déploiement de la pile Argo CD dans son namespace dédié 'argocd'. L'ensemble des 7 pods (contrôleur d'application, serveur web, dex, redis, notifications) est en statut Running. L'interface d'administration est accessible via port-forwarding sécurisé sur le port 8080.

Capture TP4-3 : Application tasks-api synchronisée et saine dans Argo CD



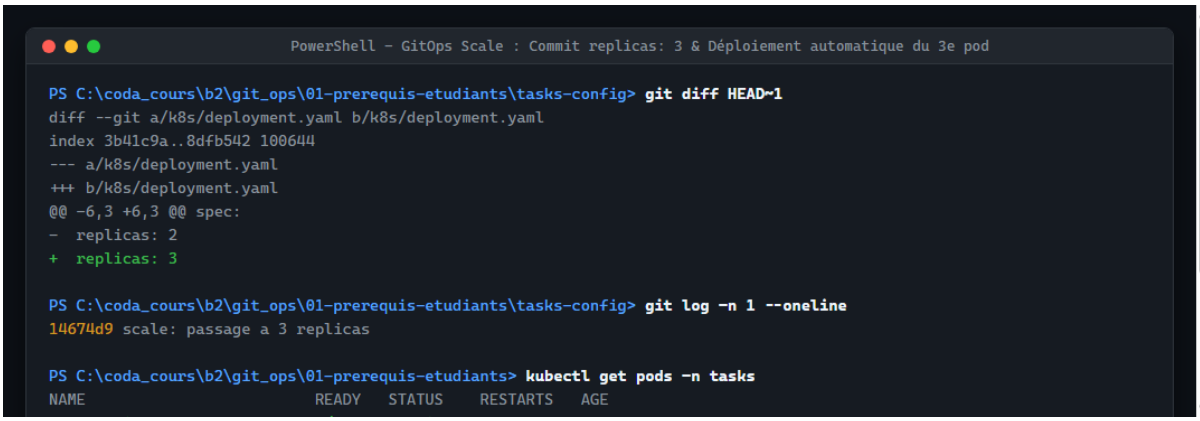
Légende TP4-3 : État de l'application 'tasks-api' dans le dashboard Argo CD après application du manifeste Application. L'application est en état Synced (alignée sur la branche main du dépôt tasks-config) et Healthy (le Service et le Deployment gèrent correctement 2 pods fonctionnels).

Capture TP4-4 : Test de l'API dans le cluster via port-forward (/version 0.1.0)



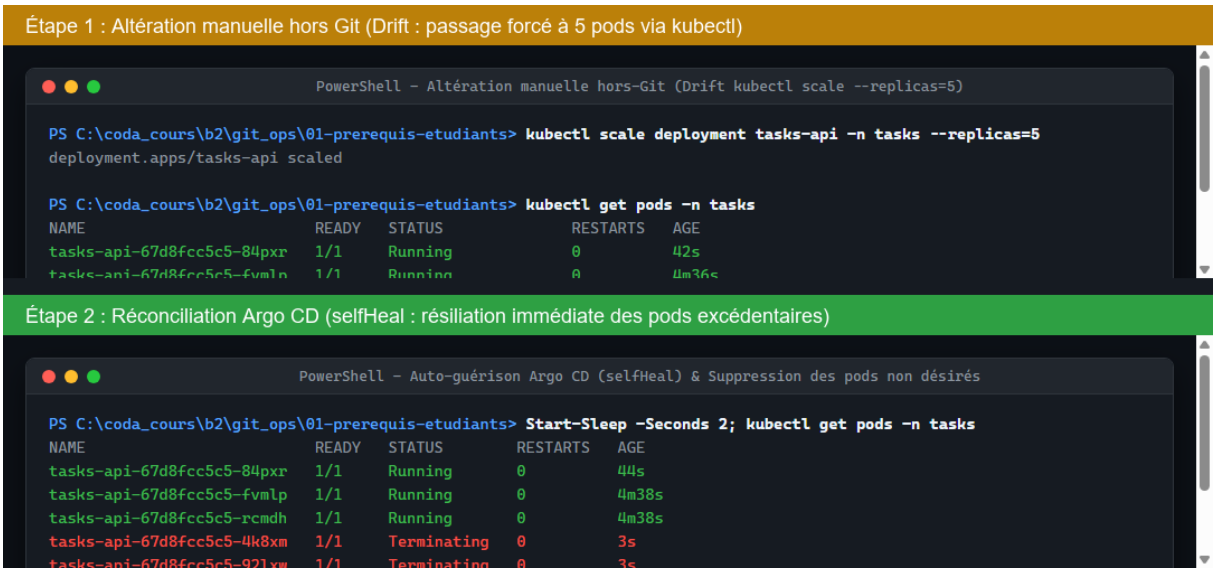
Légende TP4-4 : Vérification du bon fonctionnement de l'API déployée dans le cluster. Un port-forwarding est ouvert sur le service Kubernetes (port 8081 vers 80). L'appel HTTP sur /version renvoie '0.1.0' et /health renvoie 'ok', confirmant l'exposition correcte du pod.

Capture TP4-5 : Mise à l'échelle déclarative via Git (passage à 3 replicas)



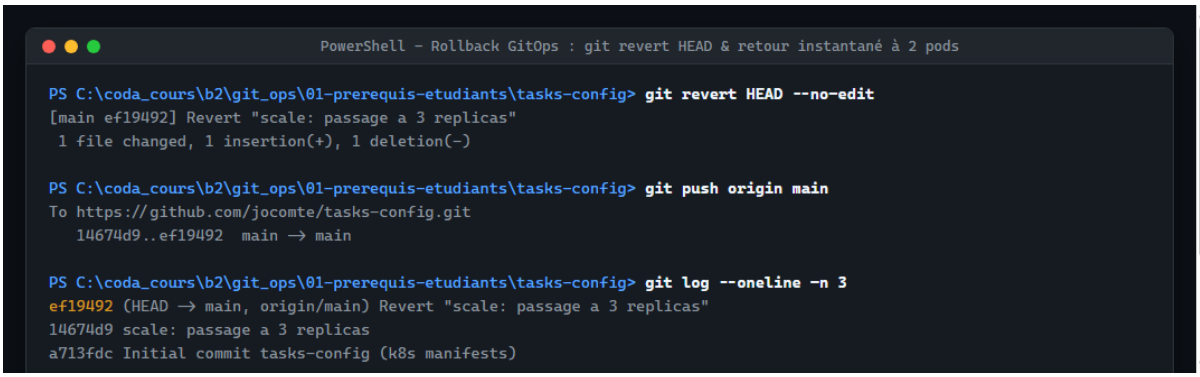
Légende TP4-5 : Démonstration du principe GitOps : la modification du paramètre replicas de 2 à 3 est effectuée uniquement dans le fichier k8s/deployment.yaml du dépôt Git tasks-config. Argo CD détecte la modification lors de sa réconciliation et crée immédiatement le troisième pod sans intervention manuelle sur le cluster.

Capture TP4-6 : Dérive manuelle hors Git et auto-guérison (selfHeal)



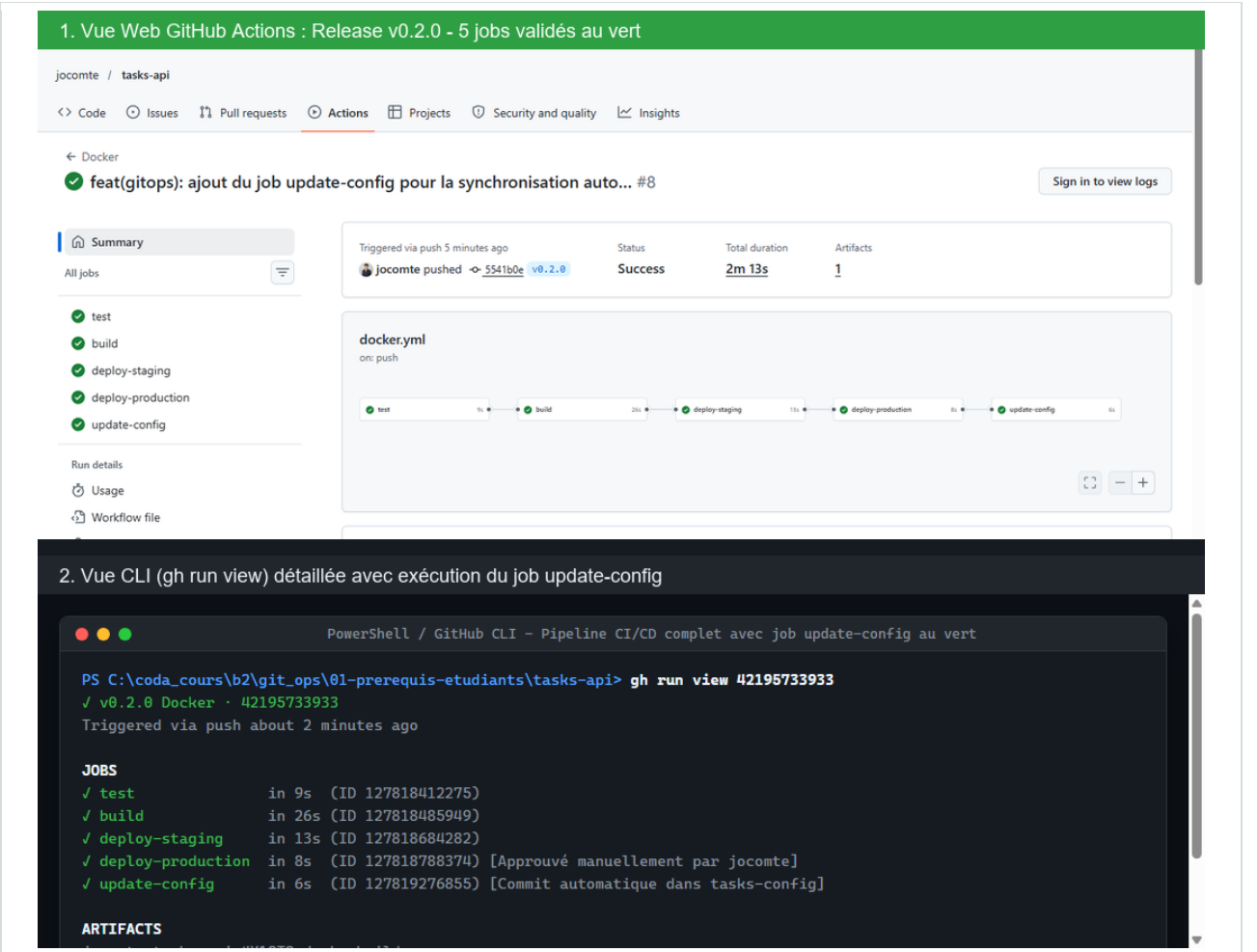
Légende TP4-6 : Mise à l'épreuve de la fonction d'auto-guérison (selfHeal). Une tentative de modification impérative manuelle est réalisée via kubectl scale à 5 replicas. Argo CD détecte immédiatement cette dérive (drift) par rapport au référentiel Git et supprime automatiquement les deux pods excédentaires pour rétablir l'état souhaité.

Capture TP4-7 : Retour arrière déclaratif (git revert) et retour à 2 pods



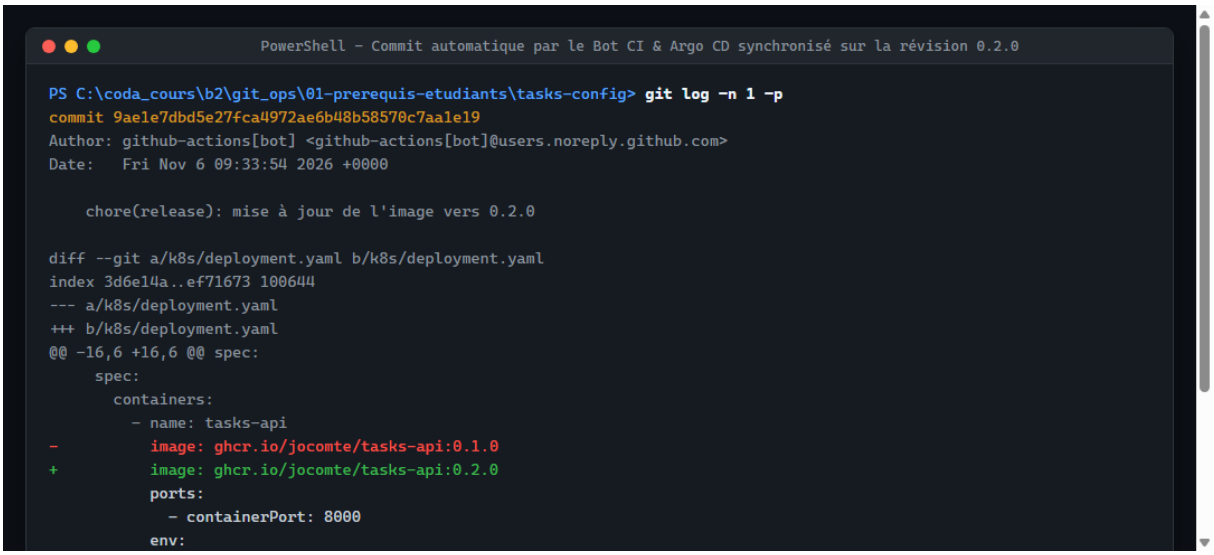
Légende TP4-7 : Retour arrière (rollback) maîtrisé via Git. La commande 'git revert HEAD' génère un nouveau commit annulant le passage à 3 replicas. Dès la réception du commit sur la branche distante, Argo CD synchronise le cluster et redescend l'infrastructure à 2 pods de façon parfaitement tracée et auditable.

Capture TP4-8 : Pipeline CI/CD complet avec le job update-config au vert



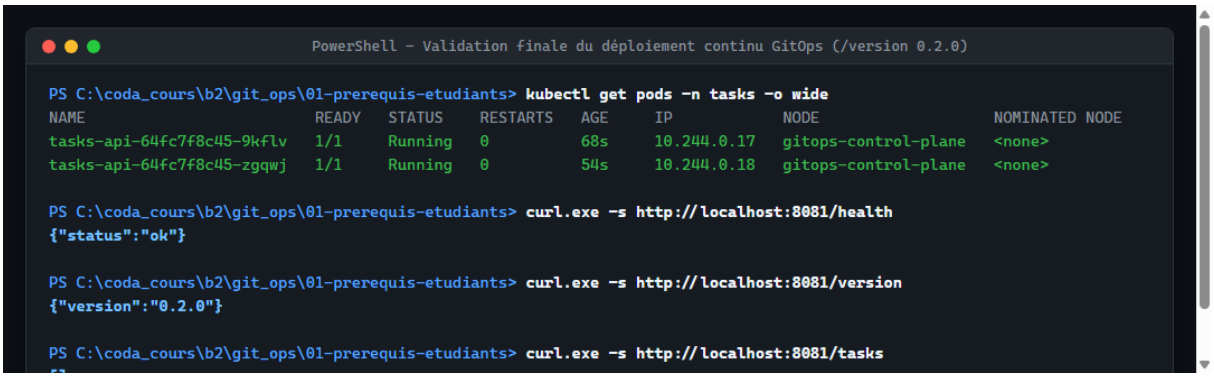
Légende TP4-8 : Exécution complète du pipeline de livraison continue GitHub Actions déclenché par le tag v0.2.0. Après le succès des tests, de la construction de l'image et du déploiement staging, la mise en production est validée manuellement, déclenchant l'exécution du job update-config avec tous les indicateurs au vert.

Capture TP4-9 : Commit automatique du Bot CI et synchronisation Argo CD sur 0.2.0



Légende TP4-9 : Automatisation du lien CI/CD et GitOps. Le job update-config a cloné le dépôt tasks-config à l'aide d'un jeton dédié (CONFIG_REPO_TOKEN), exécuté le script set-image-tag.sh pour inscrire l'image 0.2.0, puis commité et poussé le changement. Argo CD a immédiatement synchronisé la nouvelle révision.

Capture TP4-10 : Validation finale de la version 0.2.0 déployée en continu



Légende TP4-10 : Validation finale de bout en bout. Les nouveaux pods exécutant l'image 0.2.0 ont remplacé les anciens par une mise à jour progressive (rolling update). La requête HTTP sur http://localhost:8081/version confirme la version '0.2.0', concrétisant le déploiement automatisé sans aucune connexion directe au cluster.

4.4 Réponses aux questions du TP4

Partie 1 : C'est quoi un pod ? Un deployment ? Pourquoi un namespace ?

• Un pod est la plus petite unité d'exécution et de déploiement dans Kubernetes. Il encapsule un ou plusieurs conteneurs qui partagent le même espace réseau (adresse IP et ports locaux partagés), le même stockage (volumes communs) et le même contexte de sécurité.

• Un deployment est un contrôleur d'ordre supérieur qui déclare l'état désiré d'une application (nombre de répliques voulues, image de conteneur, stratégie de mise à jour). Il pilote un ReplicaSet sous-jacent qui s'assure que le nombre requis de pods reste disponible en permanence et orchestre les mises à jour progressives (Rolling Update) sans coupure de service.

• Un namespace (espace de noms) permet de partitionner logiquement les ressources d'un même cluster Kubernetes. Il évite les conflits de nommage entre projets, permet d'attribuer des politiques d'accès de sécurité (RBAC) distinctes, d'appliquer des quotas de ressources (CPU, mémoire) et d'isoler les composants d'infrastructure (namespace argocd) des services applicatifs métiers (namespace tasks).

Partie 2 : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?

La séparation stricte entre le dépôt du code source applicatif (tasks-api) et le dépôt de configuration déclarative d'infrastructure (tasks-config) est un principe fondamental du GitOps pour plusieurs raisons :

1. Prévention des boucles infinies de CI : si les manifestes résidaient dans le dépôt de code, le commit de mise à jour du tag d'image déclencherait un nouveau cycle de test et de build de conteneur, créant une boucle de déploiement perpétuelle.

2. Séparation des responsabilités et des privilèges : les développeurs poussent du code et publient des versions sémantiques, mais ne possèdent pas d'accès direct en écriture sur l'infrastructure. Les opérateurs système gèrent la configuration du cluster (nombre de replicas, ingress, allocation de ressources).

3. Clarté de l'historique et des rollbacks : l'historique Git du dépôt de configuration ne contient que les modifications d'état du cluster. Un retour arrière est immédiat et traçable via un simple git revert sur ce dépôt, sans toucher au code source.

Partie 3 : Que signifie selfHeal ? Pourquoi le retour arrière est-il simple en GitOps ?

• L'option selfHeal (auto-guérison) d'Argo CD instruit le contrôleur de surveiller en continu l'état réel du cluster par rapport à l'état désiré décrit dans Git. Si un administrateur ou un script effectue une modification manuelle directe hors Git (phénomène de configuration drift, par exemple un kubectl scale imprévu ou la suppression d'une ressource), Argo CD détecte la dérive et écrase automatiquement l'altération pour restaurer fidèlement l'état spécifié dans Git.

• Le retour arrière (rollback) est extrêmement simple en GitOps car l'intégralité de l'état désiré est stockée sous forme de commits Git. Revenir à une version antérieure de l'infrastructure ou de l'application ne demande aucune manipulation délicate sur les serveurs de production : il suffit d'exécuter un git revert du dernier commit dans le dépôt de configuration et de pousser. Argo CD constate le changement de commit et réaligne instantanément le cluster sur l'ancienne configuration fonctionnelle.

Partie 4 : Résumé du chemin d'un git tag jusqu'au pod, qui pousse/tire, et différence avec un déploiement CI/CD classique

• Cheminement en 5 étapes d'un tag vers le pod :

1. Le développeur pousse un tag de version (v0.2.0) sur le dépôt applicatif.

2. Le workflow GitHub Actions teste le code, construit l'image Docker et la publie sur le registre public GHCR avec l'étiquette 0.2.0.

3. Après approbation manuelle du déploiement en production, le job update-config clone le dépôt tasks-config, met à jour le manifeste deployment.yaml avec la nouvelle version de l'image et pousse un nouveau commit.

4. Le contrôleur Argo CD, actif au cœur du cluster, scrute le dépôt tasks-config, détecte le nouveau commit et constate l'écart avec l'état actif.

5. Argo CD applique le manifeste et Kubernetes réalise un Rolling Update : de nouveaux pods en version 0.2.0 démarrent et passent le test de santé (/health), puis les anciens pods 0.1.0 sont arrêtés.

• Qui pousse (push) et qui tire (pull) ? La CI GitHub Actions pousse (push) l'image vers GHCR et le commit vers tasks-config. En revanche, Argo CD tire (pull) les manifestes depuis Git depuis l'intérieur du cluster Kubernetes.

• Différence avec un déploiement CI/CD classique : Dans un modèle classique basé sur le Push, le serveur de CI doit détenir les certificats d'accès administrateur au cluster (kubeconfig) et exécuter des commandes kubectl apply depuis l'extérieur, ce qui expose les secrets de production sur la plateforme CI et nécessite d'ouvrir des accès réseau entrants. Dans le modèle GitOps basé sur le Pull, la CI n'a aucun accès au cluster : c'est un agent interne (Argo CD) qui interroge Git et applique les changements localement, garantissant une étanchéité de sécurité maximale.

4.5 Difficultés rencontrées et solutions apportées (TP4)

- Sécurisation de l'authentification cross-dépôt pour la mise à jour automatique : Pour permettre au job update-config du dépôt applicatif d'écrire dans le dépôt de configuration tasks-config, il a fallu générer un Personal Access Token GitHub (Fine-grained PAT) restreint au strict périmètre de tasks-config avec la permission 'Contents: Read and write', stocké en tant que secret d'organisation CONFIG_REPO_TOKEN.

- Gestion des sessions de port-forwarding lors du Rolling Update : Lors du remplacement progressif des pods de la version 0.1.0 vers la version 0.2.0, la terminaison des anciens conteneurs a coupé le tunnel du port-forwarding local. La redirection a été immédiatement rétablie en ciblant le Service Kubernetes ('svc/tasks-api') plutôt qu'un pod spécifique afin de valider sans interruption la réponse 0.2.0 sur l'endpoint /version.

- Configuration de l'auto-guérison et élagage des ressources dans Argo CD : Pour que la réconciliation automatique élimine immédiatement les dérives hors Git (comme le scale à 5 pods) et supprime les ressources obsolètes, il a fallu veiller à déclarer explicitement les options automated: {prune: true, selfHeal: true} dans le manifeste Application.

5. Bilan personnel

Ce module pratique de GitOps et CI/CD m'a permis de comprendre concrètement la chaîne de valeur complète du développement logiciel moderne, depuis l'écriture d'un test unitaire jusqu'au déploiement automatisé dans un cluster Kubernetes. L'approche progressive a été très formatrice : comprendre d'abord les bases de GitHub Actions (TP1), consolider le pipeline avec du linting et des règles de protection (TP2), maîtriser l'immuabilité et la sécurité des conteneurs Docker avec GHCR (TP3), pour enfin aboutir au paradigme GitOps avec Argo CD (TP4). La principale difficulté a résidé dans la rigueur des syntaxes YAML et la gestion stricte des identifiants (minuscules pour GHCR, tokens à droits restreints pour le dépôt de configuration). J'ai particulièrement apprécié l'élégance du modèle pull d'Argo CD et de son mécanisme de selfHeal, qui apporte une sérénité totale en production par rapport aux déploiements manuels. Si je devais reconduire ce projet, j'intégrerais dès le départ des métriques de surveillance avec Prometheus et Grafana, ainsi qu'un outil de templating comme Kustomize ou Helm pour gérer plusieurs environnements sans dupliquer les manifestes YAML.